Design layer

the whole layer, as one document

Contents

1 Fabric 5

1.1 Foundation 5
1.2 Pending updates 6
1.3 System at a glance 7
1.4 Whole model and identities 8
1.5 Agent authoring and model port 10
1.5.1 Typed construction 10
1.5.2 Model requests and conversation 11
1.5.3 Validated original input 12
1.5.4 Attempt and token bounds 13
1.5.5 Typed final answers 15
1.5.6 Failure classification and correlation 15
1.6 Agent lifecycle and action batches 15
1.6.1 Per-action states 16
1.6.2 Recovery and retained results 16
1.7 Tools and effect settlement 17
1.7.1 Tool definition and binding 17
1.7.2 Replay and cancellation 17
1.7.3 Late settlement window 18
1.7.4 Reading action outcomes 18
1.8 Policy and authenticated approvals 19
1.8.1 Admission decisions 19
1.8.2 Proof identity and reviewer audit 19
1.8.3 Answer and expiry semantics 20
1.8.4 Business identity across delayed approval 20
1.9 Graph authoring and serial control 20
1.9.1 Definition and node binding 20
1.9.2 Activity state model 21
1.9.3 Compatibility manifest 21
1.9.4 Graph bounds 22
1.9.5 Serial execution transition 22
1.10 Signals and durable wait deadlines 23
1.10.1 Typed signal consumption 23
1.10.2 Absolute deadlines and arming 24
1.11 Managed children and family budgets 24
1.11.1 Managed agent and subgraph 24
1.11.2 Ancestry, cancellation and direct handles 25
1.11.3 Root reservation ledger 25
1.12 Structured forks and joins 26
1.12.1 Membership and dispatch 27
1.12.2 Join and stop causes 27
1.12.3 Fork deadline and idle discovery 28
1.13 External jobs and cancellation ownership 28
1.13.1 Receipt and observation 28
1.13.2 Owned stop progress 28
1.13.3 Scheduled observation 29
1.14 Provider adapters and compiled composition 30
1.14.1 Generation and classification 30
1.14.2 Workflow, MCP and identity recipes 30
1.15 Protocol-neutral invocation 31
1.15.1 Service and idempotency 31
1.15.2 Durable admission and outcome incorporation 31
1.15.3 Wait and cancellation outcomes 32
1.16 Durable store and runtime ownership 33
1.16.1 Record port and acknowledgement 33
1.16.2 Lease authority 34
1.16.3 Store supervision and draining 34
1.16.4 Memory and directory boundaries 35
1.16.5 Effect timing 35
1.17 Recovery discovery and retention 35
1.17.1 Root registration and sweeper 35
1.17.2 Projection semantics 35
1.17.3 Family-safe pruning 36
1.18 Operating a worker 36

1.18.1 Startup ... 36
1.18.2 Health and progress ... 37
1.18.3 Recovery of delayed work ... 38
1.18.4 Resolving uncertain effects ... 38
1.18.5 Stopping or rolling a worker ... 38
1.18.6 Accepted turns and outcome incorporation ... 38
1.18.7 Format upgrades ... 39
1.18.8 Retention operation ... 39
1.19 Persistence compatibility and operational limits ... 40
1.19.1 Record envelopes ... 40
1.19.2 Operations and global capacity ... 40
1.19.3 Telemetry delivery ... 40
1.20 Verification and extension obligations ... 41
1.20.1 Evidence boundaries ... 41
1.20.2 Validation tools ... 41
1.20.3 Static tooling and measurement limits ... 42
1.20.4 Oracle comparison ... 42
1.20.5 Backend and callback extension ... 44
1.20.6 Retained experiments and application evidence ... 44
1.21 End-to-end walkthrough ... 45
Glossary ... 45

Fabric

1 Fabric

1.1 Foundation

Typed business decisions and explicit effect ownership govern execution.

GOAL

Run bounded typed agents and conditional graphs

Applications combine models, typed tools, human decisions and external work while retaining useful outcomes across process loss. Agents choose tool calls; graphs implement application-owned control protocols.

GOAL

Preserve the meaning of interrupted work

A restart retains accepted results and recorded routes. Effects that may have happened require evidence before execution can continue.

NO-GOAL

Retrieval and remote agent platforms

Retrieval, vector stores, document loading and splitting remain external tools. Remote Agent Protocol execution and a general middleware chain are outside Fabric.

NO-GOAL

General channel graphs and automatic compensation

Exact LangGraph APIs, time travel, arbitrary runtime code installation, shared sibling writes, quorum joins and general channels are outside the graph contract. Saga owns independent workflow compensation.

NO-GOAL

Distributed Erlang and directory power-loss durability

Shared database leases coordinate capable stores. Fabric does not coordinate through Erlang distribution or promise durable directory entries across operating-system crashes and power loss.

INVARIANT

Effects follow confirmed ownership transitions

The runner releases an effect only after the corresponding record transition and required family grant are confirmed. Revision checks serialize writes; incarnation and attempt checks reject stale reports.

mechanism

INVARIANT

Unknown outcomes retain their evidence

An uncertain effect blocks further dependent progress. Cancellation and expiration close business routing while retaining already performed results and unresolved child or external-job cleanup.

mechanism

INVARIANT

Approval applies to the exact current action

Answers match run, action or activation, requirement and approval revision. Approval rechecks current policy with fresh context and accepts proofs only from the configured verifier.

mechanism

INVARIANT

Business callbacks honor their declarations

Selection, acceptance, codecs and request preparation are pure. Replay declarations truthfully identify effects safe to repeat; application version changes accompany changes of meaning.

convention

PRINCIPLE

Keep authority at the owner

Fabric owns execution and its durable records. Applications own domain state, authorization choices, truthful effect classification and resource supervision. Provider and protocol packages retain their own contracts.

PRINCIPLE

Bind native values before collecting operations

Each tool and node retains its own input and output relationship. Heterogeneous collections hide that relationship behind a bound invocation rather than a universal application value.

PRINCIPLE

Share proven mechanisms without erasing lifecycles

Agent and graph controllers share stores, fencing, supervision, family budgets and approval semantics. They retain different control and result models. ADR-0001 ADR-0002

1.2 Pending updates

build **Streaming agents retain unresolved lifecycle scope** 2026-10-06

Map llm_wire progress to agent progress with explicit approval interruption points, partial transcript commits, backpressure, cancellation and stream closure. No public agent progress stream implements this scope.

ADR-0008

build **Whole-run time and family token bounds remain unbuilt** 2026-10-06

Per-call bounds and graph wait deadlines do not implement a whole-agent elapsed deadline. Per-agent observed tokens do not implement a shared family token ledger. Clock, approval pauses, child inheritance, restart and cleanup semantics require a decision before these become APIs.

ADR-0008

build **Approval edit and respond need continuation semantics** 2026-10-06

Approve, reject and expiry are implemented. Editing an action or responding with a substitute value remains retained intent; neither may reuse an approval for changed arguments.

ADR-0008

ruling **Historical forks and reducer equivalence need a ruling** 2026-10-06

Intended scope includes historical review forks and typed DAG reducer correspondence. Graphs use serial state and isolated forks, while their public scope excludes time travel. Resolve this scope tension and effect ownership before claiming reducer equivalence or admitting review forks.

ADR-0008

build **Suspension inside Saga needs an owned continuation** 2026-10-06

The Saga tool recipe handles bounded workflow reports and settlement. A suspending Fabric run inside Saga and durable external-child compensation authority remain separate integration contracts. No bridge package is selected.

ADR-0008

ruling **Queue-driven sweeper ownership remains unspecified** 2026-10-06

A Grind job can carry a run reference. Replacing independent store recovery with queue-driven scheduled wakeups requires an explicit sweeper ownership contract. Existing transactional delivery can bridge application acceptance to invocation without replacing Fabric recovery.

ADR-0008

verify **Child context and direct recovery need targeted evidence** 2026-10-06

Recorded intent asks for narrower typed child context and direct child recovery with consistent parent attachment. Current agent delegation shares the context type; graph context factories have a different contract. Verify the direct public child path before claiming that registered root recovery closes every case.

ADR-0008

build **Context compaction remains deferred intent** 2026-10-06

Specify application-visible compaction or summarization, provider replay compatibility and stored transcript meaning. Exhaustion stops the run and never silently discards context.

ADR-0008

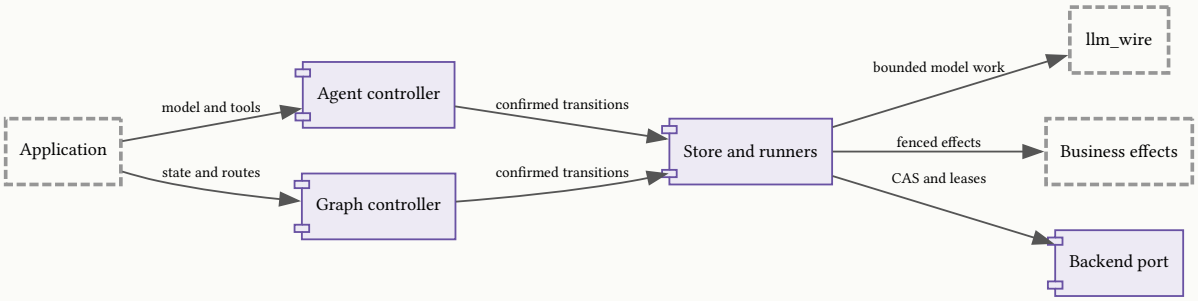
verify **Operating and verification claims need a code check** 2026-10-17

The operating-a-worker and verification sections, the graph and agent bounds tables and the failure classification restate rules from retired operations, verification, oracle and usage guides. Only the record versions, the Saga recipe length and the toolchain floor were checked against source. Check each remaining value and rule against the code and the check scripts.

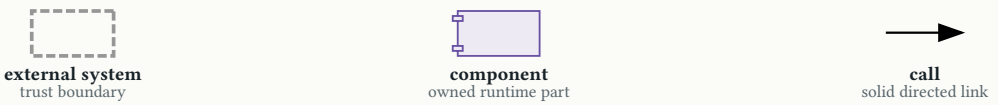
1.3 System at a glance

One durable authority serves two control models and explicit external ports.

VIEWPOINT RUNTIME · ALTITUDE L1 · BOUNDARY Application composition around Fabric



LEGEND



Violet nodes belong to Fabric. External owners keep their independent lifecycles. The database adapter is a separate package, not a second core control authority.

A Run carries durable state; Context supplies live application authority.

- An agent owns a transcript and action batches. A graph owns typed application state, committed activation receipts and chosen routes. Neither is a queue.
- The store supervises live runners. An idle approval or wait is data; dropping a handle does not stop its run.

- The application owns HTTP clients, authenticating verifiers and external services. Fabric does not start or stop a supplied http_gun client.

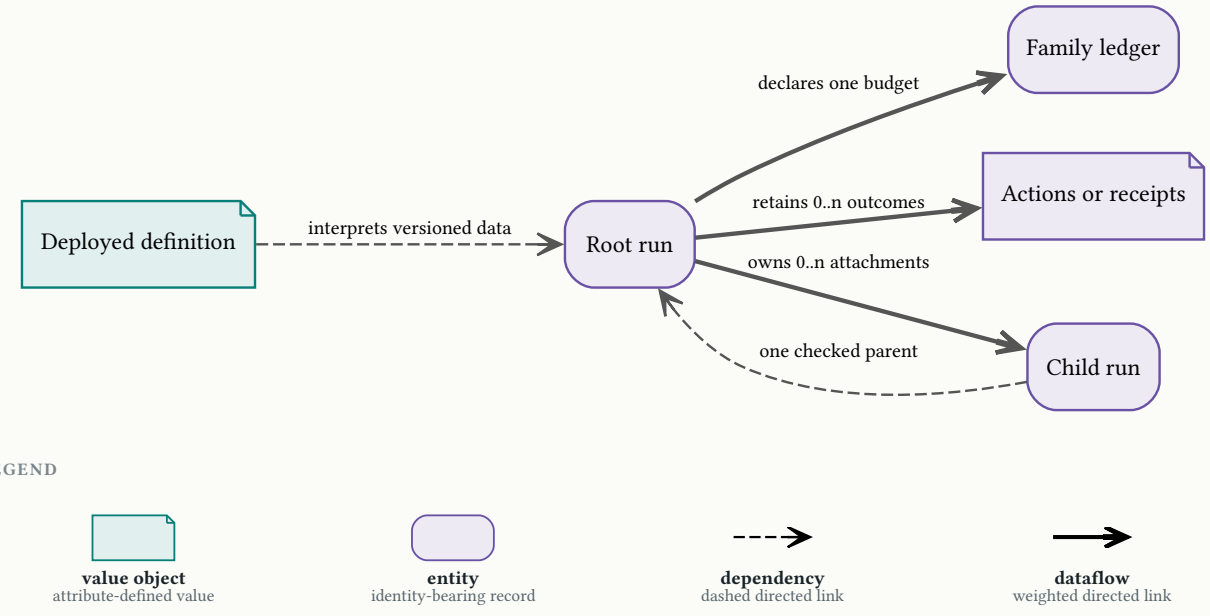
Boundary	Authority	Contract
Provider	llm_wire	Wire interpretation, schema admission and provider replay data. Fabric owns run admission and attempt budgets.
JSON	json_blueprint	Native typed boundary codecs. Schema requirements apply to model declarations and typed answers, not to every internal value.
Observations	sinal	Synchronous delivery or explicitly selected bounded forwarding. Observations do not decide execution.
PostgreSQL	fabric_postgres	Separate leased backend with application-owned pool. Its nested layer owns SQL and schema details.
Saga / Relay / Hellhound	Application recipe	Workflow evidence, MCP translation and authenticated proof wiring remain explicit. Core imports none of them.

ADR-0003 ADR-0004

1.4 Whole model and identities

Logical work, execution ownership and observation correlation have separate identities.

VIEWPOINT DOMAIN-MODEL · ALTITUDE L2 · CONTEXTS Run family and committed evidence



Each edge states ownership or retained data. Child records and the budget ledger remain separate CAS aggregates.

A Definition Version declares meaning. An Incarnation fences lost-owner reports, while Correlation only joins observations.

Run record

aggregate · stateful · execution · owned by Fabric controller through store CAS

The durable aggregate serializing one execution.

ATTRIBUTES

- authored **Identity** · RunId
  The caller selects a stable id before start; Fabric derives the id of a managed child from its parent attachment. Existing input under another definition or original input is a reuse conflict.
- authored **Definition** · DefinitionId and graph manifest
  Deployed code and native codecs interpret the retained data.
- derived **Revision and incarnation** · Positive integers
  The backend revision serializes commits. Incarnation fences work from a lost owner.
- derived **Correlation and root** · Correlation and RunId
  The first accepted correlation survives reopening; exact root follows validated ancestry.

RELATIONSHIPS

- `1 : 0..n` A run contains action records or graph receipts and may own many child attachments. Every child has exactly one retained parent.

Definition

value-object · immutable · authoring · owned by application

An immutable configuration retaining typed callbacks and boundary codecs.

ATTRIBUTES

- authored **Name and version** · DefinitionId
  Name is nonblank and version positive. A new meaning needs a new version.

Receipt

value-object · immutable · graph · owned by graph controller

One accepted graph result and its route.

ATTRIBUTES

- observed **Result and route** · Encoded input/output/state, Next or Finished or Stopped
  The immutable record of an accepted observation governs later progress.

Proof

value-object · immutable · approval · owned by configured Approvers instance

Evidence of an authenticated requirement check.

ATTRIBUTES

- observed **Verifier, requirement and lifetime** · Verifier identity, Requirement and time
  The configured verifier asserts authentication and authorization. The runtime checks issuer identity, requirement equality and proof age.

Family ledger

aggregate · stateful · admission · owned by budget CAS

The monotonic reservation aggregate for a root family.

ATTRIBUTES

derived **Claims** · Work and child grants
Validated claims reconstruct usage. A missing initialized ledger is data loss, never fresh capacity.

Identifier	Meaning	Reuse rule
RunId	1–128 ASCII letters, digits, hyphen or underscore	parse_id validates external text; new_id uses 128 random bits. Agent child ids from record version 9 and graph child and fork member ids hash their parent and work identity with a domain prefix; agent writers before version 9 keep parent-id-and-sequence child ids. A child id grants nothing without the reciprocal attachment. ADR-0019
ActionId	Model turn and provider call id	Provider call id alone can repeat; action references also name the run.
Graph activation / attempt	Visit / authorized replay	New cycle visit changes activation. Replay preserves activation and increments attempt.
Parent	AgentParent / GraphParent / GraphBranch	Tag keeps action and activation meanings distinct.
Approval reference	Run, original work identity, requirement and approval revision	Stale and already answered requests cannot authorize another action.
Signal reference	Run, activation, attempt and signal definition	One committed wait; duplicate acknowledgement compares exact result bytes.
Correlation	Observation join value	Independent of idempotency; first keyed start keeps it.

APPLICATION-DERIVED IDS NEED CANONICAL FRAMING

run.id_from_parts joins ordinary fixed-shape parts with hyphens and documents that one part containing a hyphen may alias several parts. It hashes invalid or long joined ids. Invocation and action idempotency keys use framed tuple encodings before hashing. Applications must preserve their established mappings and avoid treating correlation or attempt as logical identity. ADR-0003

1.5 Agent authoring and model port

A pure configuration is validated before a run starts.

An Agent retains its model, tools and native answer contract.

1.5.1 Typed construction

Agent definition

responsibility

A Spec retains name/version, model, bound tools, explicit policy, typed final answer, sub-agent declarations and all run bounds.

interface

agent.new creates Spec(context, String); with_answer gives its answer codec. Value-first settings retain configuration. agent.build returns the validated Agent or every ConfigError.

interactions

Tool registration checks duplicate names and schema availability; model adapters receive the same admitted input schemas. Sub-agent construction checks identities and bounds.

invariants

Building performs no effects or process allocation. A validated Agent is the only agent definition accepted by start, open and recover.

failure

Invalid names, versions, duplicate tools, missing schemas, unsupported answer/configuration and out-of-range limits remain construction failures. Runtime policy can still require approval without an approver; every answer is then refused.

1.5.2 Model requests and conversation

Configured generation model

responsibility

Apply application-selected generation controls while Fabric owns request assembly.

interface

llm.configured_model accepts the borrowed HTTP client, llm_wire Config, model identity and generation.Settings. llm.model delegates with default settings.

interactions

Every model turn applies the captured settings, then Fabric's tool declarations and native answer contract. Correlation, transcript conversion and provider replay remain Fabric-owned.

invariants

Settings are immutable per model instance and cannot replace messages, tools, output codecs or credentials. Reconstruction supplies fresh live model configuration and reapplies its settings. No settings are added to execution or history records.

failure

Invalid or provider-incompatible controls fail during llm_wire.prepare before HTTP execution. Fabric maps the refusal to InvalidRequest. The borrowed client remains application-owned.

BEHAVIOR

Configured model preserves generation controls across turns

Given An agent uses a configured generation model.

When Fabric assembles a model turn, including after reconstruction.

Then The model's generation settings apply to the request.

Then Fabric retains transcript, tool, answer and correlation authority.

level: boundary area: Model integration

- Applications retain responsibility for selecting stable model settings across deployment and recovery. Changing them is a live configuration change, not an automatic persisted-record migration. ADR-0015

Model port

responsibility

A Model is a plain application callback over a request and typed reply. Fabric owns transcript evolution and attempt accounting.

interface

model.Request carries run, turn, correlation, system prompt, messages, tool declarations and optional answer schema. Replies distinguish final answer, tool request, refusal and truncation.

interactions

fabric/llm adapts llm_wire caller-owned conversations and propagates correlation per turn. A deterministic callback supplies the same port for consumers and tests.

invariants

Assistant provider data and ordered call/results survive record encoding. Missing usage remains absent. Tool-call ids must correlate exactly within the returned batch.

failure

Opaque ModelError preserves retry classification and provider delay. Only unreachable, timed out, rate limited and overloaded are retryable; malformed requests/replies and crashes are host outcomes. Refusal and output truncation retain separate run outcomes.

Lossless history storage

responsibility

Restore application-owned message values without coupling conversation storage to execution records.

interface

fabric/history.codec returns json_blueprint Codec(List(model.Message)). Its independent envelope carries format fabric.history.v1 and messages. Each message uses a closed tagged union; all object fields are required, with explicit null for absent replay metadata.

interactions

Applications compose the codec into their own native records and transactions. It serializes every message list, including empty lists and partial generated suffixes. input.new separately validates conversation relationships before admission.

invariants

Restoration preserves message and call order, roles, text, raw arguments_json, call ids and names, provider_id, provider_state and assistant ProviderData format/value. Opaque strings remain uninterpreted; absence differs from a present empty string. The history format evolves independently of execution-record versions.

failure

Malformed shapes, tags, types, missing fields, unknown fields and unsupported format versions fail decoding rather than silently dropping information. Blueprint owns schema composition, typed codec errors and configurable JSON parsing limits.

BEHAVIOR

Stored history restores exact message values

Given An application retains any ordered message list, including a partial turn.

When The application serializes and restores that history.

Then Every message field and its order remain identical, including opaque replay data and explicit absence.

Then Restoration grants no conversation admission or execution authority.

level: boundary area: History storage

BEHAVIOR

Unreadable history refuses restoration

Given Stored history has an unsupported format or a malformed, missing or unknown field.

When The application restores that history.

Then Restoration returns an error without yielding a partial message list.

level: boundary area: History storage

ADR-0014

1.5.3 Validated original input

Original input belongs to one run.

History-aware agent input

responsibility

Admit application-owned history as context for one new agent run.

interface

input.new(history, prompt) returns an opaque Input or a typed InputError. input.messages exposes its validated ordered messages. fabric.start_with_input accepts Input; fabric.start preserves the single-prompt interface. fabric.matches_initial compares an Input against the stored original input. fabric.generated_messages returns only messages appended by this run.

interactions

Construction validates the complete message sequence before appending the current UserMessage exactly once. The stored initial boundary separates imported history and current prompt from generated messages. Recovery retains that boundary.

invariants

Original-input comparison preserves ordered message roles, text, tool-call arguments and results, and opaque provider replay data. It excludes generated messages, correlation, live context and credentials. Imported calls and results grant no tool execution or approval authority.

failure

Malformed roles, unmatched or duplicate call results, unresolved historical calls fail at input construction. Opaque provider data remains uninterpreted by the generic input boundary. An older writer that cannot retain imported history refuses admission before model or tool effects.

- Applications own conversation storage, turn selection and domain rules. Input is one execution value rather than a reusable conversation resource.
- Historical assistant answers spend no current answer attempts. The new run's model, tool, policy, token and answer bounds apply only to its new execution, including after recovery.
- Generated messages retain their exact order, call/result relationships and provider replay metadata. Fabric's existing answer API separately decodes and validates the final native answer; generated-message extraction introduces no second decoder.

BEHAVIOR

Historical calls carry no current execution authority

Given Validated original input includes a completed historical tool exchange.

When A new run starts or recovers from that input.

Then The historical exchange remains model context.

Then Only newly admitted actions can execute under current policy and bounds.

level: boundary area: Agent input

BEHAVIOR

History does not spend fresh answer attempts

Given Original input includes earlier assistant answers.

When The new run receives an invalid final answer before or after recovery.

Then Only answer attempts generated by the new run determine whether a corrective attempt remains.

level: boundary area: Agent input

ADR-0013

1.5.4 Attempt and token bounds

Bound	Default	Setting	At the limit
Model attempts	8	with_max_turns	The run ends BudgetExhausted(TurnLimit). The initial call, continuations, retries and corrective answers each consume an attempt.
Final-answer attempts	2	with_answer_attempts	The run ends AnswerInvalid. One corrective turn is the default, and each answer spends a model attempt.

Bound	Default	Setting	At the limit
Model call	600 seconds	with_model_timeout	The call stops with a retryable timeout that spends a turn. Fabric bounds custom models as well as llm_wire calls.
Tool body	60 seconds	with_tool_timeout, tool.with_timeout	The body stops; the action is uncertain unless its replay contract permits another start.
Tool result	1 MiB	with_max_result_bytes	The run fails with OutputEncodingFailed naming the limit. The bound does not stop a handler allocating a larger native value.
Concurrent tool bodies	4	with_max_concurrency	Later tools queue. Idle approvals consume no body slot.
Policy decision; runner command wait	5 seconds each	with_policy_timeout, with_command_timeout	A policy that does not answer stops the run closed with PolicyFailed. A command that waits for a busy runner returns RunnerBusy.
Model retry delay	200 ms, doubling	with_model_retry_delay	A provider Retry-After longer than the delay is waited instead, capped at ten minutes. Cancellation interrupts the wait.
Sub-agents; depth	4; 1	with_max_children, with_max_depth	The delegation is refused and the model observes the reason.
Approval request	7 days	with_approval_expiry	The stored absolute expiry rejects the action and the model observes the rejection. Records without an expiry keep none.
Approval proof	60 seconds	with_proof_lifetime	The answer is refused with ProofExpired.
Tokens; family budget	Opt-in	with_token_budget, with_family_budget	Exhaustion ends BudgetExhausted(TokenLimit or FamilyLimit). Unreported provider usage is unverifiable, never zero.
Replays of an interrupted tool body	None	tool.with_replay	The action is an uncertain effect.
Drain window	25 seconds	store.with_drain	The supervisor kills the runner; running tools become uncertain.

- Admission refuses tools whose result cannot reach another permitted model turn. Exhaustion retains outstanding work without authorizing new effects.
- Per-run token totals accumulate observed input/output usage over every reply. A configured token limit with missing usage ends as UsageUnreported.

ADR-0004

BEHAVIOR

Every attempted call spends a model turn

Given A run has a finite positive attempt budget.

When A model call is attempted, including a retry or corrective answer.

Then Its attempt count increases.

Then Exhaustion prevents another call or tool effect whose result would require that call.

level: interface area: Agent budgets

1.5.5 Typed final answers

Final-answer validation

responsibility

The agent validates a model final answer using its retained native codec.

interface

with_answer changes the public answer type through Agent, Run, Status, Snapshot and child completion. Plain agents complete with String.

interactions

The provider receives an admitted schema. Fabric validates final answers after tool use even when the provider claims schema compliance. Non-object answers may be wrapped for provider constraints.

invariants

Completed contains the decoded native answer. Invalid content never masquerades as a completed business value. A corrective turn retains the refused answer and useful validation reason.

failure

AnswerInvalid retains final raw text and reason when answer attempts or other budgets cannot fund another correction. A new incompatible reader codec can expose a previously stored answer as invalid; effects remain performed.

1.5.6 Failure classification and correlation

- Every fabric operation returns one Error, classified as NotFound, Refused, Retry, Unavailable or Incompatible. Refused carries a run state such as an expired approval or an existing start. Retry marks a transient conflict. Unavailable marks a store or runner failure whose write outcome may be unknown. The graph runtime returns its own Error under the same classification.
- Model failures are opaque values exposing a kind, a retryable flag and the provider retry delay. Each union that may gain variants has a classification or description function, so callers branch on the classification instead of on every variant.
- A run has one correlation, chosen at start or derived from its id. It is stored with the run and carried in every telemetry event of the run and its descendants, every model request, every tool call and every operation invocation. Each event also names the family root. A record without a stored root derives its exact root from readable ancestors; where an ancestor is unreadable, the topmost readable ancestor stands in and a root_inferred event reports the inference.
- Crash and exit evidence is durable and reaches callers through statuses, snapshots and invocation facts, so it names only the class, the exception or exit kind and at most five module, function, arity and line frames. It never prints the raised terms, which can carry the values a tool body, operation, policy, model call or backend was called with, including credentials in the context. The raw term reaches only the application's logger, as a debug event in the domain fabric.crash that the default configuration drops. An operation output that its codec cannot encode is retained as evidence without the native value.

1.6 Agent lifecycle and action batches

The controller emits effect descriptions; the runner commits them before execution.

Each Action retains an individual outcome inside the agent batch.

Agent controller phase awaiting-model · acting · stopping · ended · never-started

Agent execution aggregate · stateful · agent · owned by agent controller

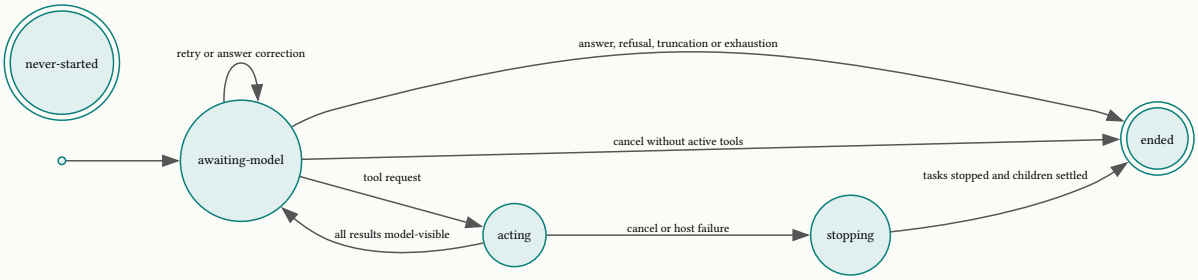
The agent-specific state of a run record.

ATTRIBUTES

derived **Phase** · Agent controller phase agent-phase · agent-lifecycle
Controls model admission, action settlement and terminal evidence.

observed **Transcript and history** · Ordered messages and ActionRecord values
Current batch and older actions retain provider data, approvals, replay counts and child links.

STATE MACHINE Agent control across replies and stopping



Never-started is an independently inserted child tombstone when cancellation wins before creation. It is vacant execution, not a successful model turn.

1.6.1 Per-action states

State family	Meaning and transition
Queued / Running	Allowed waiting slot / confirmed body-start fence. Recovery may recheck queued work; running work becomes uncertain unless declared replay-safe.
AwaitingApproval	Requirement, revision and optional expiry; approve rechecks policy. Changed requirements revoke the earlier permission and issue a new question.
Succeeded / ToolFailed	Native success or truthful Explain failure rendered to the model.
Denied / Rejected / InvalidArguments / UnknownTool / LimitReached	Model-visible refusal without a started body.
Uncertain / Reconciled	Uncertain stops continuation. Reconciliation accepts the original result once.
Delegated / ChildSettled	A child owns execution / terminal child evidence settles a finished parent's delegation without resuming it.
NotStarted / Faulted	Withdrawn before effects / host could not retain or complete the action.

Action batch

responsibility

Coordinate independently typed calls from one assistant reply while retaining call order in the transcript.

interface

ToolCall values, ActionId(turn, call_id), per-action ActionState, answered approvals, replay counts and optional child id.

interactions

Decode arguments before policy. Allowed calls enter a bounded executor; each result commits independently. Other calls may finish while one requires approval or reconciliation.

invariants

A report must name the expected action and current ownership. Repeated or foreign reports cannot progress state. The next model turn waits until every necessary action is model-visible.

failure

Codec and policy faults are host failures. Unknown tools, invalid arguments, truthful business failures and explicit denials are visible observations. Uncertainty never becomes an ordinary model-visible failure automatically.

1.6.2 Recovery and retained results

- A lost model task can start another model attempt under remaining budgets. Already accepted assistant turns and tool results are reused.
- A queued approved action asks for approval again after ownership loss because its checked context was live and is unavailable. A stored answer grants no later incarnation authority.

- A started action without result retains uncertainty unless its explicit replay promise permits a bounded restart. Replayed tool count is retained.
- A child's approvals and uncertain actions appear through the parent family. Public commands route references to their named child rather than inventing a parent action.

BEHAVIOR

A partial batch retains independent outcomes

Given Some tool results are committed and another body has started.

When The owning runner is lost and the run is recovered.

Then Committed results remain recorded.

Then The interrupted body is uncertain unless its retained replay contract permits re-execution.

Then No earlier successful tool is repeated solely because the batch was incomplete.

level: interface area: Agent recovery

1.7 Tools and effect settlement

Typed application handlers supply truthful failure and replay classification.

The Effect Fence precedes the body. A Replay Contract alone permits bounded re-execution of an interrupted body. An Uncertain Effect requires Reconciliation or authoritative Settlement.

1.7.1 Tool definition and binding

Typed tool boundary

responsibility

A Definition owns tool name, description and input/output codecs. Binding retains the typed handler and its application error classifier before collection as Tool(context).

interface

define, bind and bind_settling; handlers receive live context, tool.Call(run, action, correlation) and decoded input. Definition accessors serve consumer wiring and policy.input.

interactions

The same definition supplies the model declaration and execution decoder. Generic handler errors become Explain or Uncertain only at this boundary; they need no persistence error codec.

invariants

Input authorization may additionally occur inside the handler. A model name or schema is not authentication. A declaration is not effect permission.

failure

Explain asserts a definite business failure; Uncertain asserts an effect may have happened. Handler crash, task loss and timeout conservatively retain uncertainty. Output encoding faults after an effect do not permit retry.

1.7.2 Replay and cancellation

Effect contract

responsibility

Keep at-most-one local admission distinct from external effect guarantees.

interface

with_replay explicitly authorizes bounded re-execution and with_timeout sets the tool body bound. idempotency_key is stable for logical run/action.

interactions

Executor obtains a confirmed start fence before the body. Cancellation stops local tasks and the controller waits for declared late settlement or child outcomes.

invariants

Neither a timeout nor a read-only annotation proves a mutating operation safe to replay. Downstream idempotency, transaction or fencing must uphold the declared contract.

failure

Stop before the fence is definite NotStarted. Stop after the fence can remain Uncertain even when the local task no longer runs. Terminal cancellation preserves effect history.

1.7.3 Late settlement window

Settlement port

responsibility

Accept the eventual result of an effect whose local invocation task stopped.

interface

bind_settling supplies Settlement and a finite settle_within. settle reports confirmation, AlreadyRecorded, NotAwaited or an unconfirmed store outcome.

interactions

A stopping run first confirms its executor stopped. A settlement offered while the task may still act waits within the bound; settlement from its own task is refused immediately. A running run with an uncertain action can accept a later definite settlement.

invariants

The first valid settlement or reconciliation wins. A terminal settlement adds evidence without restarting a model or graph successor.

failure

Too-late settlement leaves a person responsible for uncertainty. An uncertain settlement cannot resolve an already uncertain active action. Lost storage acknowledgement must not be described as accepted settlement.

1.7.4 Reading action outcomes

- A Completed run means the model produced an accepted final answer. It does not mean every action succeeded: a denied reservation can end in a completed run whose answer reports the refusal.
- A snapshot exposes each action record by action id and recorded call. The record state distinguishes Succeeded, Denied, ToolFailed and Uncertain, and a failed snapshot read stays distinct from a failed action.
- The application decides which actions matter. Requiring every exploratory tool to succeed differs from requiring one payment to succeed. For an external effect the application also consults its authoritative effect record, because a schema-valid model answer is not that record.

BEHAVIOR

A completed run can contain a denied action

Given An agent calls a tool and policy denies the call.

When The model then produces a valid final answer.

Then The run completes with that answer.

Then The action record states Denied.

level: boundary area: Action outcomes

BEHAVIOR

Cancellation retains unknown external effects

Given A body may have acted and its outcome is unresolved.

When Cancellation or expiration closes execution.

Then Business continuation is suppressed.

Then Unresolved evidence remains inspectable and reconcilable.

Then A stop request or local task exit does not claim the remote effect stopped.

level: interface area: Effect lifetime

ADR-0002 ADR-0005

1.8 Policy and authenticated approvals

Permission is checked against exact work and fresh live context.

Policy decides current admission. Approval names a pending question; Proof supplies authenticated evidence for its answer.

1.8.1 Admission decisions

Policy gate

responsibility

Authorize each agent call/delegation and graph activity, signal, job, owned job, child or fork admission.

interface

Policy(context) receives Action(run, Step, name, arguments_json, Target), with decoded input represented by its original codec. Decision is Allow, Deny(reason) or RequireApproval(Requirement).

interactions

Application chooses policy; always_allow is explicit. Agent denial becomes a model-visible observation; graph denial becomes a failed graph. External policy callback is bounded.

invariants

Policy failure and invalid requirement fail closed. Visibility of advertised tools and permission to execute remain different application decisions; no general dynamic visibility policy is supplied.

failure

Policy error, exception or timeout never substitutes Allow. A changed runtime configuration is not permission to reinterpret already retained arguments.

1.8.2 Proof identity and reviewer audit

Approver port

responsibility

Authenticate and authorize one credential for a Requirement without forcing Hellhound into Fabric.

interface

approvers.new(name, verify), check and with_proof_lifetime. Verifier returns a typed Reviewer or NotAuthenticated, NotAuthorized or Unavailable. Proof is opaque.

interactions

Application constructs Approvers once at boot and shares that exact value between runtime and answer handler. Hellhound recipe validates audience and application-owned approve: scope convention.

invariants

A proof must come from the configured instance, match the exact requirement and remain within the 60-second default lifetime. It is not single-use because the durable answer already commits once.

failure

NoApprovers, OtherApprovers, OtherRequirement and ProofExpired refuse the answer without mutation. The verifier callback runs synchronously in check; its author must bound network calls. A proof lifetime is not an automatic callback deadline.

1.8.3 Answer and expiry semantics

- Approve rechecks current policy and uses the same fresh context for the admitted action. Denial or policy failure wins. Another requirement returns RequirementChanged and a new question.
- Reject requires a matching proof and records the reviewer, verifier and reason. It denies the existing action; it grants no later permission.
- Approval expiry uses store-clock time and retained absolute due time. Agent expiry is a model-visible rejection; graph expiry is ExpiredApproval failure. Old records without due time remain unbounded.
- Wrong, stale, duplicate, ended and expired references have distinct typed outcomes. Answering cannot consume a different activation or command. Unavailable clock or write confirmation admits no effect.

ADR-0004

1.8.4 Business identity across delayed approval

- Action input is stored while context stays live. Work whose identity a later approval must preserve carries that identity in its typed input, such as the document, its revision and the requester, instead of taking those values from context when the approval arrives.
- Before an answer, the application reloads current permissions and document state, and policy compares the retained input with that fresh context. Fabric checks only the policy supplied. It cannot identify which application fields must remain unchanged or detect stale facts the caller supplies.
- The requester stays separate from the reviewer the approver returns. A proof authenticates the answer to its requirement; it neither replaces the action's business identity nor refreshes application permissions.
- The effect boundary rechecks the required business conditions, for example by checking the expected revision and current permission in one database statement. Stronger ordering against concurrent changes requires the application's transaction or the external service's contract.

BEHAVIOR

A changed requirement revokes the earlier question

Given An action waits for approval and a valid proof matches its stored question.

When A fresh policy check requires a different requirement.

Then The original answer does not authorize the action.

Then The caller receives the new approval requirement and revision.

level: interface area: Approval answers

BEHAVIOR

A rejected proof leaves the question answerable

Given A pending request has not expired or ended.

When An answer supplies a proof from another verifier, requirement or lifetime.

Then The answer is refused with its typed proof reason.

Then The pending action is unchanged.

level: interface area: Approval answers

1.9 Graph authoring and serial control

Typed state and declared routes define one control decision per activation.

A Graph records each Activation, its authorized Attempt and accepted Receipt.

1.9.1 Definition and node binding

Graph definition

responsibility

Bind independently typed operations into one application state protocol with native final output.

interface

definition.new(identity, entry, nodes, state codec, answer codec), node(select, accept, destinations), with_max_activations and build. Commands are Continue(state, NodeId) or Finish(state, answer).

interactions

Select projects current state to native input; accept combines state and typed output. The bound node erases private input/output types behind encoded boundary callbacks.

invariants

Construction rejects invalid ids, duplicate nodes, missing entry, unknown destinations and every operation configuration problem. Cycles are allowed but count activations; default maximum is 100.

failure

Select/accept errors and destination outside the declared set refuse transition. Initial state, selected input, next state and answer must encode and decode. Purity is an author obligation checked only indirectly by bounded callback execution.

1.9.2 Activity state model

The activity projection below covers the executable-body lifecycle. Signal, job, child and fork phases refine waiting and stopping through the following units; they do not turn a wait into an activity.

Graph activity phase ready · queued · running · approval · blocked · stopping · ended

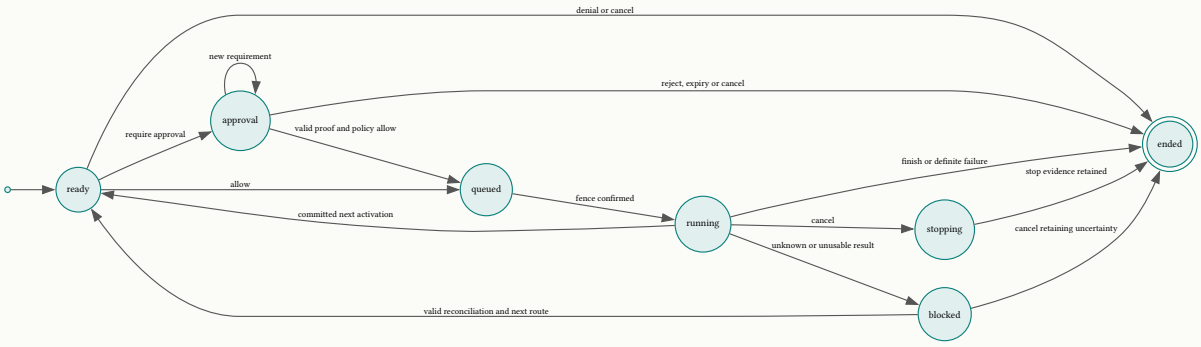
Graph activity activation entity · stateful · graph · owned by graph controller

One activity visit with retained input, attempt and result.

ATTRIBUTES

derived **Activity phase** · Graph activity phase activity-phase · activity-lifecycle
This is the activity projection of graph phase, derived from retained control state.

STATE MACHINE Activity admission, result and stop



Recovery of unfenced queued work returns to current policy inspection. Ownership loss after the fence blocks, or spends a new bounded attempt only under an explicit replay contract. A new route reserves another activation; the drawing does not collapse its identity.

1.9.3 Compatibility manifest

Deployed graph contract

responsibility

Refuse restoration under a graph that changes the interpretation of saved work.

interface

Canonical manifest of entry, node ids, operation names/versions, execution kind, allowed destinations, replay and deadline contracts; application state and answer codecs remain deployed.

interactions

Node list ordering does not alter the manifest. Recovery checks saved inputs, outputs, state, routes, membership and definition identity before effect admission; it does not rerun past select or accept callbacks.

invariants

Name, version, structural compatibility and native boundary compatibility are separate assertions. Signature cannot identify arbitrary code changes, so callback and codec meaning changes require an application version bump.

failure

Unsupported envelope, corrupt control record, changed topology, incompatible codec or forbidden saved route stops recovery. Existing successful receipts remain data and are not re-inferred.

1.9.4 Graph bounds

Bound	Default	Setting	At the limit
Pure callbacks	1 second	with_callback_timeout	The run fails with CallbackFailed.
Activity body	60 seconds	with_operation_timeout	The body stops and the run blocks on an uncertain effect.
Runner command wait	1 second	with_command_timeout	The command returns RunnerBusy; a cancellation is committed to the record instead.
Approval request	7 days	with_approval_expiry	The run fails with ExpiredApproval.
Signal, job, child or fork wait	7 days	operation.with_deadline	The run fails or expires with DeadlineExpired after stopping owned children.
Activations	100	with_max_activations	The run ends Exhausted.

- Every timeout is a duration, and a timeout that may be lifted takes an explicit infinite value. The store clock sets and judges every deadline, so every node judges it alike.
- A setter only stores its value, because a bound can come from configuration. Build reports every out-of-range bound at once, naming its setter, its value and its minimum and maximum.

1.9.5 Serial execution transition

Graph controller

responsibility

Own current activation, receipt sequence, encoded application state and effect descriptions.

interface

Ready → Queued → Running for activities; separate phases retain approval, arming, signals, jobs, children, forks, blocked effects and stopping.

interactions

A successful body returns typed output, then pure acceptance proposes state and route. One CAS commit records output receipt, state, route, budget changes and next prepared activation before successors run.

invariants

Each completed activation has one receipt. A retry preserves its ordinal and increases attempt. Live reports match incarnation, activation and attempt. Exhaustion retains the next unstarted work.

failure

Explain stops as OperationFailed; Uncertain blocks. Invalid output, encoding or transition after an effect retains unusable output separately from reason for reconciliation. Cancellation retains a returned result with Stopped route and bypasses business acceptance.

Internal phase	Work owned
Ready / Queued / Running	Inspection / admitted unfenced / started activity.

Internal phase	Work owned
AwaitingApproval / ArmingWait	A question / admitted wait whose absolute due time is not yet confirmed.
WaitingSignal / WaitingJob / StoppingJob	Committed external input / observation / retained stop progress.
Joining / WaitingChild / ChildBlocked / StoppingChild	Managed child observation, idle attachment, unresolved child and cleanup.
PreparingFork / Forking / WaitingFork	Membership preparation, active scope and parked scope.
Blocked / Stopping / Ended	Uncertain or invalid result, local activity stop and terminal outcome.

BEHAVIOR

A committed route survives different later inference

Given An activation receipt and its next route have committed.

When The runtime restarts with a decision producer that would now answer differently.

Then Recovery follows the retained route.

Then The earlier producer is not called to choose it again.

level: interface area: Graph recovery

BEHAVIOR

An unconfirmed transition releases no successor

Given A result proposes next state and a route.

When The write acknowledgement cannot be confirmed.

Then No successor effect is released.

Then Recovery must inspect the exact original write or report its unknown outcome.

level: interface area: Graph commits

ADR-0001 ADR-0002

1.10 Signals and durable wait deadlines

Waits retain correlation data and release execution capacity.

A Signal is input to one committed wait, whose deadline survives recovery.

1.10.1 Typed signal consumption

Signal port

responsibility

Accept one typed external value for a committed wait and atomically apply its graph decision.

interface

signal.new(versioned output contract), operation.await_signal(input codec), graph.deliver and deliver_json. SignalReference carries run, activation, attempt and contract.

interactions

Policy gates publishing a wait. Waiting holds no executor body or permanent runner. The application authenticates/authorizes a signal caller independently of the reference.

invariants

Delivery validates original saved contract and deployed codec. Receipt, new state, route and successor commit together. Identical consumed delivery is acknowledged from retained exact bytes; changed whitespace is a changed payload.

failure

Early unsolicited values are not buffered. Wrong/stale contract, conflicting duplicate, cancelled wait and malformed value are command refusals. Failed decoding/acceptance does not consume a signal.

1.10.2 Absolute deadlines and arming

Wait deadline

responsibility

Bound one admitted signal, job, child or fork using a restart-stable backend time domain.

interface

operation.with_deadline(run.After(Duration) or Infinity); seven days is default for new waits. Activities use body timeout instead and cannot declare wait deadlines.

interactions

Admission and approval precede clock sampling. Arming commits an absolute due time before publishing the wait or creating a managed child. Recovery preserves it. Discovery makes overdue waits eligible after downtime.

invariants

Time is UTC Unix milliseconds from the lease/discovery clock. Clock errors never fall back on another node clock. Wall-clock correction may advance or delay eligibility; samples need not be monotonic.

failure

Expiration closes routing first, retains its cause and settles owned work. It cannot undo an effect. Time samples and revision commits are separate: a commit may cross the due time after its last eligibility sample.

BEHAVIOR

A duplicate consumed signal is acknowledged once

Given A reference has a committed receipt for a delivered encoded value.

When The identical reference and exact bytes are delivered again.

Then The existing receipt is acknowledged.

Then Acceptance and successor scheduling do not run again.

level: interface area: Signal delivery

BEHAVIOR

Expiration suppresses late business routing

Given An admitted wait reaches its retained due time.

When A delivery, observation, reconciliation or recovery attempts acceptance.

Then Expiration records its cause rather than accepting a new business route.

Then Already known results and unresolved owned cleanup remain retained.

level: interface area: Wait deadlines

ADR-0005

1.11 Managed children and family budgets

A durable attachment grants ownership without blocking on another run.

A checked Attachment establishes child ownership. One Family can contain both controllers and share retained Family Grant values under an optional Family Budget.

1.11.1 Managed agent and subgraph

Child attachment

responsibility

Compose an ordinary agent or subgraph as typed graph work while retaining its own durable record and lifecycle.

interface

graph.as_subgraph and graph/agent.Definition/new/as_operation expose native input/output adapters. Parent references distinguish agent action, graph activation and fork member.

interactions

Parent reserves child identity and attachment before start; child retains the reciprocal parent. Lost start acknowledgement adopts the reserved id after parent checks; graph children also check identity and initial input. Lost completion wakeup reads committed child state.

invariants

A record under a reserved child id is the child only when its parent link names exactly that action, activation or member and the parent names it back. Adoption, reattachment, cancellation, stopping cleanup and lookup through the parent refuse any other record as lost child evidence, which leaves the delegation uncertain. Child creation uses the same store; its business effects pass current policy and bounded ancestry checks. Existing child keeps its native controller. Parent cannot call a waiting child completed or settled merely because it parked.

failure

Missing reserved child can still be created. Missing acknowledged child is data loss and cannot be recreated. Incompatible definitions, mapping failure, child approval and uncertainty preserve separate states.

1.11.2 Ancestry, cancellation and direct handles

- Before model admission, tool fences, graph effect fences and child starts, verify exact saved attachment through at most 64 ancestry links. A closed, unrelated, unreadable or excessive chain grants no new business work.
- Parent cancellation records intent, cancels admitted children and inserts a terminal never-started tombstone for a reserved absent child. It retains ownership until settled; approval and reconciliation cannot reopen a closed ancestor.
- A parent can park and release its runner and lease while a child waits. Local notifications are hints; discovery observes changed child revisions, including all unsettled fork members.
- Terminal child effects remain evidence. Explicit stored settlement can update a finished parent's history without resuming execution; child uncertainty must be resolved at the child before parent acceptance.
- Legacy missing exact root walks the parent chain. The topmost readable ancestor fallback emits root_inferred and is never persisted as an exact root.

Child deadline cleanup

responsibility

Close parent acceptance at the attachment deadline while preserving the actual child outcome.

interface

Retained due time, child identity, stop reason and ChildSettled or ChildUnresolved disposition.

interactions

Expiration records cause before requesting cancellation and uses already reserved cleanup authority. Observation may settle after due time without rearming the business wait.

invariants

Completion that loses the parent deadline race remains evidence with no parent route. Once a stop cause commits, cleanup does not require another deadline clock sample.

failure

Uncertain effects retain family and dependency-only discovery. Missing child or incompatible binding refuses settlement rather than fabricating absence or another child.

1.11.3 Root reservation ledger

Family budget

responsibility

Bound cumulative work admissions, child starts and nesting across mixed agent/graph families.

interface

budget.limits(work:), with_children and with_depth; agent/graph with_family_budget declares limits only on a root. Limits allow zero to deny new work; depth is at most 63.

interactions

One separate CAS ledger prevents quota writes changing the execution revision. Graph attempts reserve before policy; agent models reserve before call; agent tools reserve at their fence; managed child identity/depth reserve before creation.

invariants

Exact repeated claim adopts its grant without another charge, even at capacity. New model attempts, replay attempts and cycle visits spend capacity. Failed, unused and uncertain grants are not refunded implicitly.

failure

Unconfirmed grants release nothing. Changed child depth conflicts. Missing or corrupt initialized ledger and changed limits fail closed. Five conflict retries end as contention, not unlimited capacity.

- Root bootstrap stores declaration initialized=false, creates or adopts the ledger, then confirms initialized=true before execution. An initialized missing ledger cannot be replaced. Cancellation during bootstrap seals bookkeeping without restarting work.
- Only roots declare a family budget. Descendants inherit through saved ancestry and cannot reset limits or depth. Budgeted starts require agent writer v7 even for graphs that may later create agents.
- Work counts admission, not token charges, global service load or concurrent body slots. Cleanup consumes existing obligations rather than fresh business grants.

ADR-0005

BEHAVIOR

A repeated grant does not create fresh capacity

Given A logical work or child claim is already in the root ledger.

When The exact claim is retried after acknowledgement loss or recovery.

Then The original grant is acknowledged.

Then Usage does not increase again.

Then Changed identity meaning or child depth is refused.

level: interface · area: Family admission

1.12 Structured forks and joins

Children own private state; the parent joins retained results in declared order.

A Fork fixes membership before dispatch and retains private child outcomes.

Fork member lifecycle pending · reserved · active · uncertain · succeeded · failed · cancelled · withdrawn · rejected

Fork member

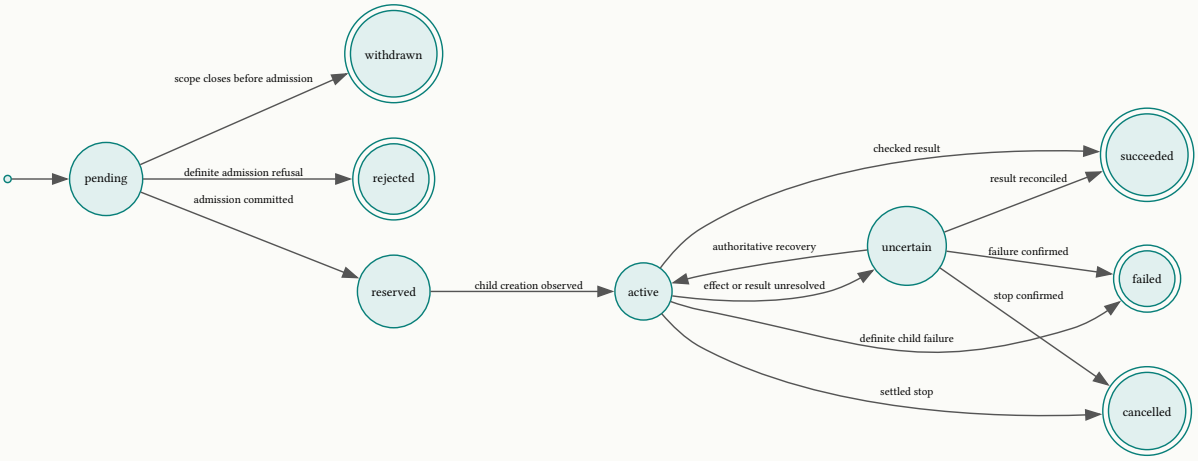
entity · stateful · graph · owned by graph fork scope

One ordered child request within an activation-scoped occurrence.

ATTRIBUTES

observed **Progress** · Fork member lifecycle fork-phase · fork-lifecycle
State retains the difference between admission refusal and an admitted child failure.

STATE MACHINE Member admission and authoritative settlement



Reserved and active members consume capacity. The structural model keeps terminal observations immutable; contradictory terminal evidence is refused.

1.12.1 Membership and dispatch

Fork occurrence

responsibility

Fix ordered membership for a pair or bounded map before admitting a member.

interface

graph.both keeps two native answer types; graph.map takes one child runtime, max_members and concurrency. Each member has a one-based ordinal and private child record.

interactions

Equal inputs are distinct members. Parent run, activation and ordinal determine stable child identity. Recovery checks fixed membership against saved input and deployed typed binding.

invariants

Empty map succeeds without children. Oversize map fails before admission. Admission follows order and counts admitted unsettled children, including approval/signal/uncertainty waits, against concurrency.

failure

Uncertainty blocks new admissions and joining; an unavailable observer never frees capacity. A stopped scope withdraws pending members and keeps admitted cleanup.

1.12.2 Join and stop causes

Typed join

responsibility

Accept one parent result after every required member has authoritative settlement.

interface

Typed ordered pair/list success or settled fork.Failure with failed member reference; branch opens retained member by activation and ordinal.

interactions

A definite rejection, failure or unexpected child cancellation closes admission and retains the first scope cause. Parent explicit cancellation/expiration separately suppresses fallback business acceptance.

invariants

Finish timing cannot change result order. Identical terminal observations are idempotent; conflicting ones are refused. Saved results survive restart without rerunning children.

failure

Invalid join retains scope and encoded result for reconciliation. Supplied result must agree with retained members. Missing acknowledged children are data loss. Deadline or parent cancellation prevents a new route even after all results arrive.

1.12.3 Fork deadline and idle discovery

- The admitted absolute deadline covers preparation, membership admission, execution and join acceptance. Approval time precedes arming. Expiration before membership owns no children.
- Before admission and after join callbacks, clock checks can close routing. Expiration withdraws pending members and cancels admitted unsettled members with their same identities.
- A parked scope has no unacknowledged starts and cannot admit or join immediately. Discovery watches every unsettled child revision; lost notifications and independent descendant recovery cannot reopen parent business work.
- After stop commits, dependency-only discovery follows cleanup without an already-expired timestamp causing immediate repeat claims. Streaming partial joins, shared sibling writes, races and quorums are excluded.

ADR-0001 ADR-0005

BEHAVIOR

Equal map inputs remain distinct work

Given A map contains equal encoded inputs at different positions.

When The scope prepares and admits members.

Then Each position retains its own ordinal and child identity.

Then The join returns one outcome per position in declared order.

level: interface area: Fork membership

1.13 External jobs and cancellation ownership

Submission, observation and cancellation evidence retain different meanings.

An external Job remains independent of the local observation lifetime.

1.13.1 Receipt and observation

External job port

responsibility

Observe a previously submitted external job to a checked business outcome.

interface

graph/job defines a retained receipt/output contract and typed progress; operation.await_job is read-only, own_job also binds a stop request and failure classifier. Submission is a separate fenced activity.

interactions

Application adapter chooses Saga, Grind or another service and stable logical submission key. Manual poll_job is default; a retained positive poll interval uses leased discovery.

invariants

Submission acknowledgement proves accepted submission, never business completion. Retrying a logical submission requires downstream idempotency or authoritative lookup. A receipt grants only its declared ownership.

failure

Lost submission receipt without lookup is uncertain. Read failure keeps waiting; a later completed outcome commits once and no further remote read is needed for that result.

1.13.2 Owned stop progress

Progress	Evidence	Allowed follow-up
Queued	Stop intent and authority committed; request not started	Recovery may execute after its fence.

Progress	Evidence	Allowed follow-up
Started	Stop body admitted and may have acted	Interrupted request is uncertain, never automatically repeated.
Accepted	Service accepted the stop request	Observe terminal outcome; this does not prove cancellation.
Refused / uncertain	Definite refusal / unknown stop outcome	Keep original receipt and inspect authority; no success route after local stop.
Remote stopped / completed / failed	Authoritative terminal evidence	Retain actual disposition and end cleanup without business acceptance.

Owned job cleanup

responsibility

Preserve a stop obligation that can outlive local cancellation or expiration.

interface

OwnedJob policy target admits cancellation ownership; StopReason is caller cancellation or retained deadline expiration.

interactions

Commit stop cause before requests. Request callback passes its own confirmed fence and reuses the existing work grant even after ancestral business admission closes.

invariants

Later local stop does not replace first committed cause or erase uncertainty. A completed remote artifact is retained if completion wins; stop acknowledgement never overwrites it.

failure

Unresolved cleanup retains family. Manual or scheduled authoritative observation resolves accepted/refused/uncertain stop state. No routing callback runs after cancellation/expiration.

1.13.3 Scheduled observation

Polling schedule

responsibility

Rediscover due observation without permanent tasks or VM timer authority.

interface

Observer with retained interval; discovery Poll trigger may also carry an absolute deadline. The first ready claim is immediately due.

interactions

Backend claims free eligible runs atomically with lease, wait key, dependency revision and claim time. Concurrent scanners claim disjoint work. Recovery revalidates saved root and contracts.

invariants

Next claim is at least the interval after successful prior claim. Manual observation does not move this automatic schedule. Metadata refresh preserves same-key schedule time.

failure

Scan interval, batches, unavailable callbacks and lease expiry can delay observation; no completion-latency SLA follows. Failed claim or stale projection supplies no effect permission.

BEHAVIOR

Stop acknowledgement is not terminal cancellation

Given An owned external job has a committed local stop cause.

When The service acknowledges a stop request.

Then The request acceptance is retained.

Then The job remains owned until authoritative terminal evidence settles it.

Then No business route is resumed.

level: interface area: External job cleanup

ADR-0005

1.14 Provider adapters and compiled composition

Substantial lifecycle behavior lives in core; application mappings remain recipes.

The Classification Wire fixes receipt interpretation independently of fresh execution context. A Recipe translates an independent package's outcomes into Fabric ports.

1.14.1 Generation and classification

LLM adapters

responsibility

Bind independent llm_wire generation and classification capabilities to Fabric model or operation ports.

interface

fabric/llm.model, fabric/llm.configured_model, graph/llm.decision and graph/classify.decision with typed input, fixed questions/wire and fresh request callback.

interactions

Application supplies an http_gun.Client and reveal-closure credentials. Generation retains provider receipts and typed answer codec; classification retains labels, distributions, rubric, model, optional usage and evidence.

invariants

One classification operation version fixes the pure wire and finite receipt bounds. Fresh live settings come only after policy admission and never enter durable data. Offline receipt decoding invokes neither network nor credential reveal.

failure

Incompatible live byte limits above receipt bounds fail in typed preparation before credentials or I/O. Lower limits never prevent older receipt decoding. NotSent is Explain; possibly-sent or completed provider failures conservatively become Uncertain at graph operation boundary.

- An endpoint override must implement the version's fixed protocol. Switching protocol requires another operation version, with compatible earlier deployed definitions retained.
- Confidence is optional concentration evidence; it is not probability that the chosen business answer is correct. Thresholds, route fallback and evaluation belong to the application.
- Inference, routing and admission are three decisions. A model can select admitted enum routes or bounded task lists; it cannot install code or bypass policy.

ADR-0006

1.14.2 Workflow, MCP and identity recipes

Consumer composition

responsibility

Translate each package outcome explicitly while preserving its independent owner.

interface

Retained compiled consumers for Saga tool, Relay typed tools/discovery/graph/serve/run-id, and Hellhound approvers. The module documentation describes each recipe and names the consumer module that compiles it.

interactions

Saga reports complete typed workflow outcome or operational error; consumer selects safe Explain/Uncertain projection and late settlement. Relay invocation renders native answer/evidence and run metadata. Hellhound verifies authentication without owning Fabric policy.

invariants

Core depends on llm_wire, http_gun, json_blueprint and sinal; no Saga, Relay, Hellhound or Grind dependency. Adapters do not acquire universal retry/error/durability meanings.

failure

MCP arbitrary schema/name admission is separate from listing. Remote read-only hints are application trust assertions, not proof a mutating effect is safe. Discovery preserves absent structured output separately from explicit null.

RECIPES REMAIN EXECUTABLE EXAMPLES

Recipes compile and run as consumer tests. Recipe length remains a review responsibility without line ceilings. Shared pinned Snips synchronizes root README excerpts from named regions in compiled examples; their compilers and tests validate behavior. See ADR-0018. Core owns no typesafe, saga or relay package. ADR-0003

1.15 Protocol-neutral invocation

Request ownership determines whether a bounded waiter cancels work.

Invocation binds request lifetime to a reusable agent or graph service.

1.15.1 Service and idempotency

Invocation service

responsibility

Bind agent/graph execution once and return a portable response for any request transport.

interface

invoke.agent, invoke.agent_with_history or invoke.graph creates Service; Request retains typed context/input, principal, optional key/correlation and disconnect selector. check validates a finite wait; default is 25 seconds.

interactions

Key scopes include service name, runtime family and authenticated principal. Framed tuple hashing derives run id; same key/input reopens it. Changed correlation on retry does not change first accepted run correlation.

invariants

Changed original input under the same key is key_reused, including changed imported history whose first message is unchanged. Agent services use fabric.matches_initial; graph services use graph.matches_initial. Shared services must set a stable authenticated principal; with_principal supplies identity scope and performs no authentication. Live context and cancellation selectors are never stored or compared as input.

failure

Initial responses always retain attempted run id. Invalid wait, key reuse and incompatible input refuse the request. Agent StartUnconfirmed and agent/graph StoreUnavailable during admission return OutcomeUnknown. Unavailable keyed reopen or input verification also retains OutcomeUnknown. Other admission errors remain Refused. General read/await failures return Refused with code unavailable.

1.15.2 Durable admission and outcome incorporation

Application delivery composition

responsibility

Bridge application acceptance and outcome incorporation without introducing another execution owner.

interface

An existing transactional job or outbox retains an accepted application turn and its stable service, principal, key and original input. A worker invokes the existing service; the leased store and registered sweeper own run recovery.

interactions

Acceptance and delivery publication commit together in the application's existing transaction. Fabric may begin later. After completion, an idempotent application transaction incorporates the retained answer and generated messages and records that the logical turn was applied.

invariants

Retain the Fabric outcome until incorporation commits. The application checks its durable applied marker before delivery; an applied turn is never reinvoked after Fabric pruning. Shared pools or borrowed connections do not establish atomic Fabric admission with application writes.

failure

Working remains in progress; approval/input suspension needs its declared continuation; uncertainty requires inspection or reconciliation of the same identity; ended failure and definite refusal remain separate. An absent answer alone establishes neither retry eligibility nor successful incorporation.

- The application starts no second lease-renewal loop, abandoned-run scanner or competing queue for Fabric runs. Existing delivery retries cover the acceptance-to-start interval; Fabric recovery covers admitted runs.
- An application-authored per-turn graph can own result incorporation as an explicit operation after agent completion. Its application transaction must use a stable logical idempotency key; interruption after a possible commit requires reconciliation or truthful bounded replay.
- A graph moves incorporation into retained execution but does not make its initial admission atomic with application acceptance. The existing job or outbox still covers that interval. No adapter admit_in capability follows from sharing a database connection.

ADR-0013

1.15.3 Wait and cancellation outcomes

Request lifetime

responsibility

Bound waiting while preserving the native run outcome and ownership obligations.

interface

Response accessors expose answer, id, Kind, code, description and JSON facts; Kind distinguishes Answered, Working, AwaitingApproval, AwaitingInput, OutcomeUnknown, Unattended, Ended and Refused.

interactions

Keyed work outlives the request and disconnect. Unkeyed in-flight work cancels on wait expiry or disconnect. Approval/reconciliation return immediately even for unkeyed work; application owns follow-up by id.

invariants

Cancellation failure never claims stop. Completed answer that wins a cancellation race remains Answered. Cancelled/expired runs with unresolved local or child effects expose OutcomeUnknown.

failure

Transport handler must keep invoke wait below its own request deadline. Unknown effects require status/reconciliation with original identity, not another effect submission. Unknown admission retains the attempted run id and existing error code. Retry the same principal/key or recover that run; an unknown response never authorizes a fresh logical submission.

BEHAVIOR

Unconfirmed admission preserves the attempted run

Given Admission may have stored a run but its acknowledgement or keyed readback is unavailable.

When The invocation returns.

Then The response classifies the outcome as unknown and retains the attempted run identity.

Then A retry with the same principal and key names that same logical run.

level: interface area: Invocation ownership

BEHAVIOR

Keyed retries retain the same logical run

Given A stable service, principal and idempotency key already name a run.

When A request repeats the original input with a new correlation.

Then The same run is opened.

Then The first accepted correlation remains on its work.

level: interface area: Invocation ownership

BEHAVIOR

Unkeyed wait expiry cancels in-flight work

Given An unkeyed invocation still has live work.

When Its finite wait expires or the caller disconnects.

Then Cancellation is requested for that run.

Then The response preserves confirmed completion, unresolved effects or a cancellation failure truthfully.

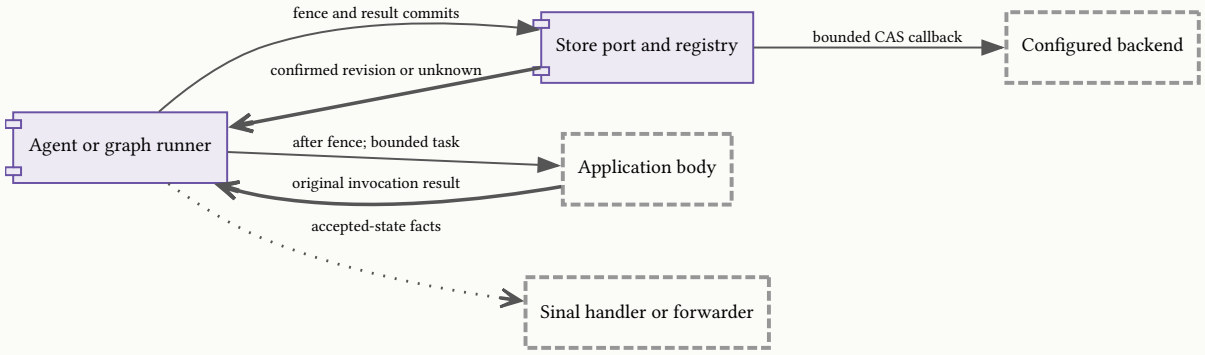
level: interface area: Invocation ownership

ADR-0003 ADR-0007

1.16 Durable store and runtime ownership

Revision CAS, leases and supervision answer separate questions.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS One runner owns isolated work and confirmed advancement



LEGEND

component owned runtime part

external system trust boundary

call solid directed link

dataflow weighted directed link

pubsub dotted publish or consume link

The runner requests a durable fence before a body. It commits the original result before publishing accepted facts. Backend callbacks and bodies are separately bounded; the observation path supplies no execution authority.

A Lease selects the execution driver. A Write Token identifies the exact attempted write.

1.16.1 Record port and acknowledgement

Backend port

responsibility

Keep the latest encoded record and revision while allowing exactly one advancement of an expected revision.

interface

Unleased get, insert and compare_and_set; leased current record/holder plus atomic Lease action. Errors are NotFound, AlreadyExists, Conflict, LeaseRefused or Unavailable.

interactions

Insert stores revision 1; CAS stores expected+1 only if expected is current. Unavailable writes are checked by exact encoded bytes at exact target revision, including fresh write token.

invariants

Two concurrent calls at one expected revision cannot both succeed across shared processes/machines. Encode once per logical write and reuse those bytes for confirmation; equivalent state from another writer is insufficient.

failure

Backend callback crashes/timeouts become Unavailable. An unconfirmed insert becomes StartUnconfirmed and may arrive later. Reuse original id or cancel_stored rather than starting another logical run.

1.16.2 Lease authority

Leased store

responsibility

Coordinate which node drives work without replacing the record revision authority.

interface

LeasedBackend get/insert/CAS, now, renew, claim_expired and claim_ready. Holder is Free or Held(owner, live). Lease mutation is Hold, Claim, Seize or Release.

interactions

Hold requires the named holder, including expired ownership; Claim permits free/same/expired owner; Seize and Release still require correct record revision. Renewal extends only live held leases and changes no execution revision.

invariants

Running fences and work-in-flight commits require retained lease authority. A final commit releases with revision check. Store renews runners around lease/3 and stops local tasks on failed or lost renewal.

failure

Live foreign lease is not recovered. Idle commands can apply without its runner when no work needs it. Foreign in-flight commands may return RunUnattended; cancellation can fence through storage. Claiming expired/ready work never grants routing without validation.

1.16.3 Store supervision and draining

Store lifecycle

responsibility

Supervise and register agent/graph runners while isolating bounded backend callbacks.

interface

store.in_memory, directory, new or leased; start/supervised, readiness, stop, with_drain and backend timeout. Store name/runtime owner is independent of the process that first starts a run.

interactions

Without automatic recovery, the application supervises store.supervised. With automatic recovery, sweeper.supervised supplies the store subtree and ordered sweeper together; it replaces store.supervised. Admission starts after readiness. Stop closes acceptance and drains all owned runners including ones still delivering synchronous observations.

invariants

Default drain is 25 seconds. Within the window a started effect can complete, its successor is saved but handed off without starting. Idle runs retain no runner. Bounded backend callbacks do not block unrelated store operations.

failure

Forced task stop or unconfirmed handoff can remain uncertain. Drain summary separately records confirmed/failed/pending handoffs and forced/other exits; unavailable accounting is explicit. Stop result never erases durable evidence.

1.16.4 Memory and directory boundaries

- In-memory records disappear when their owning store state disappears. They support deterministic testing and independent agent usefulness without database setup.
- Directory CAS uses host filesystem serialization and atomic record replacement. It can coordinate several VMs on one host, with no lease-driven multi-node ownership.
- Directory acknowledged writes survive process/VM loss within its documented filesystem assumptions, but directory entries are not fsynced. Power loss may lose a recent fence and permit re-execution; it is not production power-loss durability.
- Applications choose a database backend for production. The nested fabric_postgres layer owns PostgreSQL transactions, schema, indexes, migrations, projections and application-owned pool integration.

ADR-0005

1.16.5 Effect timing

Decision	Durable boundary	Effect released afterward
Start	Initial record and budget bootstrap confirmed	First model attempt or graph inspection.
Policy allowance	Queued transition	Executor asks for its fence.
Body start	Running fence plus grant and ancestry validation	Typed application body.
Graph result	Receipt/state/route/successor committed	Next activation.
Child admission	Reservation and exact reciprocal attachment	Create/adopt one child.
Stop	Cancellation/expiration intent committed	Local task stop and owned cleanup.
Lease loss	Ownership no longer valid	No new effect; stop local tasks and retain uncertainty.
Observation	Accepted execution commit	Sinal fact, which cannot roll back state.

1.17 Recovery discovery and retention

Scheduling and metadata guide recovery; saved execution state remains authority.

Discovery supplies scheduling evidence. Only a complete Settled Family qualifies for backend pruning checks.

1.17.1 Root registration and sweeper

Sweeper

responsibility

Find expired leases and changed/due idle work and recover through registered compatible roots.

interface

sweeper.agent rebuilds live context. sweeper.graph rebuilds the complete graph and child runtimes against the supplied pinned store. Each builder has five seconds. sweeper.supervised supplies a RestForOne subtree containing the store followed by its sweeper; use it instead of store.supervised. Script-only start links to an already started store and caller.

interactions

A boot scan precedes interval-based scans; the interval starts after each completed batch. A batch takes at most 50 expired leases and 50 changed idle dependencies without overlapping scans. Each candidate recovery has 30 seconds. Unknown identities are reported rather than executed. Mixed parent chains resolve exact root. Changed-child and due-wait projections share a bounded ready claim path.

invariants

Notifications merely wake observation. Compatible definition, original attachment, lease, quota and policy checks still govern effects. Manual read/open/await never silently seize recovery ownership. Held work awaits lease expiry; a signal wait without a deadline needs explicit delivery.

failure

Stale projections require refresh; missing children/roots, unreadable records and context-builder failure refuse recovery. Failed callbacks leave lease-expiry retry paths. The sweeper stops before the store drains, and restarts when the store restarts.

1.17.2 Projection semantics

Discovery, retention and statistics

responsibility

Extract minimal versioned metadata for backend recovery, pruning and diagnostics without executing deployed callbacks.

interface

store/discovery Wait(key, Changed/Poll/At); store/retention Metadata(run, parent, children, settled); store/statistics diagnostic projections; conformance checks backend extension guarantees.

interactions

Discovery v14 tracks approval expiry and idle work; retention v13 includes agent/graph families and budget ledger. Backend refresh updates metadata while preserving execution bytes, revision, original ages and same-key observation times.

invariants

Projection eligibility never authorizes an effect. Unknown/corrupt records do not qualify as finished, absent or safe to prune. An ended run with uncertain effects can remain unsettled.

failure

Stale metadata can block recovery or pruning until repaired. Statistics are bounded operational diagnostics, not admission health or proof of no active external effect.

1.17.3 Family-safe pruning

- A pruner must verify complete reciprocal family links, expected membership, all settled descendants, age and live-lease absence atomically. Retention.inspect alone grants no deletion authority.
- A missing acknowledged child, mismatched parent key, unknown format or unresolved owned job prevents pruning the complete family. Budget ledger links participate.
- A completed result and later compensation remain history. Deleting retained runs ends the idempotent reopening and replay-evidence window; applications own retention duration and external audit needs.
- PostgreSQL pruning, refresh/statistics and schema internals are specified by the nested adapter layer. The operating section owns startup, deployment, readiness and incident procedures.

ADR-0005

BEHAVIOR

Unknown metadata cannot authorize pruning

Given A family contains an unreadable record, missing reciprocal member or unresolved effect.

When Retention is attempted.

Then The family remains retained.

Then An ended status is not treated as sufficient settlement evidence.

level: interface area: Retention

1.18 Operating a worker

Operators read evidence from storage and the runtime; evidence never authorizes recovery or proves an external effect.

The application owns deployment, credentials, access control and alert thresholds, while Fabric owns saved execution and conditional writes. A Node names one live VM, Readiness describes the instant after a bounded backend read, and the Sweeper finds recoverable work.

1.18.1 Startup

- Each live VM has one stable node id, reused after that VM restarts and never shared by two live VMs. An id holds 1–128 letters, digits and the characters `.`, `_`, `-`, `@` and `:`, and `nonode@nohost` is refused. Generated process names alone do not distinguish VMs.
- The application starts its database pool, builds the adapter settings from that connection and node id, and applies the adapter migration before accepting runs or enabling the sweeper. A migration failure fails startup. Migration is idempotent under a database advisory lock.
- After an upgrade the application refreshes discovery, retention and statistics metadata in bounded positive batches until each returns zero. Concurrent refreshers skip locked rows, so one zero does not prove another refresher finished. Refresh preserves execution bytes, revisions, leases and record-write ages, and unknown records can remain unknown.

- Store configuration, including the record version and drain window, precedes start. The returned store value serves every handle, graph and recovery registration.
- Supervision starts the pool, the optional Sinal forwarder, the migration barrier, the store subtree, the sweeper and request admission, in that order, under a rest-for-one arrangement. The pool and forwarder outlive the store's stop. The migration barrier completes before the sweeper's boot scan.
- Every root's deployed recovery code is registered with the sweeper, which replaces the plain store supervisor. Context is rebuilt from the root id, and a graph factory rebuilds its children against the supplied pinned store. Admission opens only after readiness reports Accepting.
- The directory backend supports restart on one host and lacks power-loss durability, so production procedures use the PostgreSQL adapter.

1.18.2 Health and progress

Readiness evidence	Meaning and response
Accepting	The backend read succeeded, the factory accepts work and local runners have safe lease evidence. A fresh worker can be ready before its first renewal.
StoreDraining	Admission stops for this worker while its shutdown finishes.
RunnerFactoryUnavailable	The store cannot start runners; supervisor failures and startup configuration need inspection.
LeaseUnconfirmed	Active runners lack a safe lease window. Database reachability, pool contention and worker stalls need inspection while fencing remains automatic.
Last-success age	The local monotonic age since the start of the last confirmed renewal batch. None means no renewal has succeeded in this store incarnation, and a large age on an idle worker is not a failure.

Statistics group	Meaning and response
working	Known records needing a runner with a live lease. The group does not establish that a remote process is alive.
unattended	Known records needing a runner without a live lease. Recovery registration, sweeps and the saved state need inspection.
waiting	Unfinished records that need no runner, including legitimate signal, job and child waits. Age alone does not make a wait broken.
approval and reconciliation	Records with current approval requests or unresolved effects. These counts overlap the main groups and each other, and children count individually.
finished	Ended executions that can still need reconciliation. Finished does not mean safe to prune.
unknown	Missing, stale, incompatible, corrupt or mismatched diagnostic metadata. Unknown is never evidence that no work exists.
leases per node	Live leases by the node component of the owner token, comparable with the expected workers.
oldest overdue lease	The age of the oldest expired lease, a recovery-backlog gauge rather than recovery latency.

- A failed readiness or statistics read is a failed observation, never a healthy zero. A successful readiness read promises nothing about the next request.
- Counts measure individual records, and budget ledgers are reported separately. The oldest record age is the time since the latest durable record write, not the duration in a group; an empty group has none. Statistics use one database snapshot and one database clock sample.
- Alert thresholds derive from the application's expected waits and recovery objectives. Repeated readiness failures, a growing unattended or expired backlog, unknown records, renewal failures, failed or unmatched sweeps and forwarder drops warrant alerts. An event can be missing after process loss, so persisted state and current gauges drive investigation instead of event absence.

1.18.3 Recovery of delayed work

- Leases renew every third of their duration. The PostgreSQL default is 30 seconds and accepted durations run from 100 ms to 2^32−1 ms. A shorter lease permits quicker takeover and leaves less room for database delay and process stalls, and runners fence themselves before their safe local lease window ends.
- The sweeper scans at boot and waits its interval after each completed batch. A batch claims at most 50 expired leases and 50 changed or due idle waits, scans never overlap, and root restoration takes 30 seconds including at most 5 seconds to rebuild context. A failed or unmatched recovery leaves its claim to expire, so recovery time is lease expiry plus scan wait plus earlier work plus backend and context latency; the interval is no recovery guarantee.
- For one run the operator opens it and reads its snapshot or await result, retaining read errors as evidence. Unattended work is recovered with its compatible deployed definition. Explicit agent recovery can take the lease of an earlier local store incarnation immediately, while foreign live leases stay held. A new workflow never replaces an unattended one.
- The sweep report counts claimed, recovered, unmatched and failed roots. An unmatched root needs its exact identity and version registered, and a failed root needs its decoding, attachment, context, callback or backend error resolved. A graph-owned agent is restored through the root graph registration. Changing an identity to silence an incompatibility is not a migration.
- Legitimate waits resume through their own operation: approval answers, typed signal delivery and manual job polling. Scheduled jobs, due deadlines and changed child dependencies need a registered sweeper, and a signal wait without a deadline needs explicit delivery. After a refused approval answer the caller fetches the new request instead of retrying a stale reference.

1.18.4 Resolving uncertain effects

- The operator inspects the original operation in the external system by its durable effect identity, idempotency key or receipt. A missing Fabric result is not proof that the operation failed, and the body is never called again merely to discover whether it ran.
- An active agent's uncertain tool is reconciled with the real result and the run continues once its other waits clear. A finished agent's uncertain tool records evidence without restarting work. A finished delegation settles its descendants first and then propagates their saved outcomes, and the returned snapshot can still contain unresolved actions.
- A graph operation is reconciled with the actual output encoded by its output codec and the current reconciliation reference. After cancellation the result is retained and no route runs. Managed-child outcomes come from the actual child and reach the parent when the parent recovers.
- A saved cancellation request for an owned job proves intent, not remote termination, so observation continues until authoritative terminal evidence is saved. Cancelling a read-only job wait detaches observation and does not cancel the external work.
- The replay contract belongs to the application and its external effect, and operational reporting grants no replay permission. Unresolved families stay retained, and cancellation stops further work without erasing uncertainty.

1.18.5 Stopping or rolling a worker

- Admission and the sweeper stop before the store subtree. During drain runners start no new work, wait for running bodies and model replies, save results and hand off remaining work. The default window is 25 seconds per runner, after which the supervisor kills the runner and interrupted effects can remain uncertain. Suspended runs have no runner to drain.
- The outer shutdown budget covers the drain window, up to four more seconds for admission, accounting and observation, and the cleanup of processes, forwarder and pool. The factory stops runners concurrently. Abrupt VM or host loss yields no summary and follows lease recovery.
- The drain summary is emitted after the factory stops and before the store closes, and its metadata names the local subtree. It reports the runners present when admission closed, including admitted starts racing shutdown, and the confirmed, failed and pending handoffs, counted once per runner. A handoff counts as confirmed after a lost acknowledgement only once readback confirms it.
- The summary also partitions the cohort into killed, exited and unobserved runners, where killed records a forced exit whatever its cause, and reports the monotonic time since admission closed. A confirmed handoff and a kill can describe the same runner. Pending and unobserved counts stay explicit after the one-second accounting deadline.
- A drain-unavailable report means accounting could not be read and never implies an empty drain. The forwarder stays alive through store shutdown, and an enqueued observation does not prove delivery to an external monitor.
- After restart the operator observes readiness, sweep results and the expired backlog. Failed, pending or killed handoffs require checking saved records and external effects, and the summary never instructs Fabric to repeat an operation.

1.18.6 Accepted turns and outcome incorporation

Observation	Delivery action
Answered	Incorporate the native answer and generated messages idempotently.

Observation	Delivery action
Working	Preserve the turn identity and let execution and the existing delivery mechanism continue.
AwaitingApproval or AwaitingInput	Record the suspended observation and arrange the required application interaction.
OutcomeUnknown	Inspect admission and effect evidence and reconcile the original execution; never submit the effect under a new key.
Unattended	Inspect recovery registration, lease state and sweeper health.
Ended	Record the terminal failure or cancellation and retain required evidence.
Refused	Inspect the code: correct invalid configuration or handle key reuse. An unavailable code can describe a read or await failure after execution exists, so the original identity is retained and the run inspected before deciding.

- Admission and incorporation have separate failure windows. Admission records application acceptance before a Fabric run exists, and incorporation applies a retained outcome after the run finishes. A delivery carries a stable accepted-turn key, the authenticated principal and the complete original input, and the invocation wait stays below the delivery mechanism's deadline.
- Before invoking, the application reads its Applied Marker and returns the retained result when the turn is already applied. The marker stays effective after the Fabric record is pruned. On Answered, one idempotent application transaction applies the answer and generated messages and sets the marker, so a crash before commit leaves the outcome available and a lost commit acknowledgement is resolved by reading the marker.
- Pruning stays disabled for a store holding unapplied outcomes. The adapter's age-based prune has no application acknowledgement predicate, so a sufficiently old finished run can be deleted before incorporation, and a finite age cannot guarantee retention under unbounded delivery delay. An applied turn is never invoked again after its Fabric record disappears.
- An application-authored per-turn graph can make result application a durable operation. The operation takes history from graph state, and its retained generation receipt lets recovery retry only the unfinished application operation. Result application is an idempotent transaction keyed by the accepted turn. A crash between commit and graph save is an interrupted effect, and a replay contract is enabled only when repeating the transaction returns the same result without applying it twice.
- A running or unresolved application node keeps the family out of the settled pruning candidates, and application markers still prevent resubmission after the family is pruned. Neither option introduces a queue, a conversation resource or an atomic admission API.

1.18.7 Format upgrades

- The versions in the record-envelope table are independent, and all of them are checked before a rolling deployment. Readers that understand the target agent format deploy first while the shared older writer stays selected, and newer features and writer settings follow only after every participating reader is compatible. The writer setting is configured before stores start, is used by the sweeper, and neither rewrites old records nor reconfigures live runners. Once newer records exist, rolling back the binary alone can be impossible.
- Version 8 retains the initial-input boundary used by duplicate comparison, generated-message extraction and answer-repair accounting. Older records read with their one-prompt semantics. A writer selected below version 8 refuses imported history before starting work, and selecting an older writer does not remove history from a stored version 8 run.
- Version 9 names new delegations' child runs by derived ids. A writer selected below version 9 keeps numbered child ids, which every reader accepts, and refuses to write a derived id. Children already stored keep their ids, so one version 9 parent can hold both forms.
- Graph formats and database schema roll out separately. Older backend writers stop before schema migration 6, which replaces the scalar dependency index with the fork-aware dependency array, and the agent writer setting is neither a graph nor a database compatibility switch. Old compatible recovery registrations stay until their retained executions complete or are deliberately migrated.

1.18.8 Retention operation

- The application chooses the retention interval from its needs. After retention metadata is refreshed, pruning deletes at most a bounded number of eligible root families and returns the deleted record count including children. A lost deletion acknowledgement leaves the count unknown, and repeating prune is safe.
- Long-retained finished families are investigated through their saved uncertainty and attachments. Child rows and phases are never edited to make a family prune, because children and budget ledgers are not independently disposable.

1.19 Persistence compatibility and operational limits

Versioned bytes carry data; compatible deployed code supplies meaning.

1.19.1 Record envelopes

Envelope	Read / write contract	Retained meaning
fabric.run	Reader accepts versions 1–9; configurable writer 2–9, current 9	Derived child run ids, transcript and initial-input boundary, assistant provider data, actions, approvals, budgets, mixed parents, child settlement and exact family root. An older writer refuses data it cannot represent.
fabric.graph	Reader accepts 5–15; current writer 15	Manifest, initial/state/answer values, ordered receipts, phases, signals, job ownership, deadlines, child/fork scope and family declaration. Old no-deadline records keep none.
Family ledger	Independent versioned budget record	Validated monotonic claims and immutable limits; initialized root marker distinguishes retryable bootstrap from lost data.
Backend schema and projections	Schema 7; discovery 14, retention 13, statistics 3	Migrations precede new backend writers. Each projection is refreshed after an upgrade, and stale metadata is never execution evidence.

- Formats validate tags, counters, contiguous receipts, route linkage, phase identities, approval correlation and version-dependent fields. A future envelope returns unsupported rather than best-effort reinterpretation.
- Legacy classifier receipts and provider replay turns decode offline under compatible retained operation definitions. Live credentials and strict new transport limits are never historical decoding inputs.
- Rolling deployments choose an agent writer that every deployed reader supports and refuse capabilities older writers cannot represent. Graph format compatibility is independent; downgrading agent writer does not downgrade graph records.
- Agent v8 retains the initial-input boundary. Versions 1–7 infer one original UserMessage for a started run and zero for an empty terminal record. Empty v1–2 cancelled children retain their never-started interpretation; supported v3–7 empty terminal records retain their exact terminal outcome. Their readers remain supported; their writers refuse imported history rather than silently discarding its meaning.
- Recovery validates the initial boundary and its transcript prefix before using it for input equality, generated-message extraction or answer-attempt accounting. Unsupported future versions remain unsupported.

ADR-0006 ADR-0013

1.19.2 Operations and global capacity

- Run-local concurrency and family cumulative work bounds do not provide global service admission/backpressure. Applications bound simultaneous requests, callback captures, HTTP pools and external capacity.
- Handles retain current context and bound closures. A handle is not a process or durable credential; releasing it does not release application resources or cancel stored work.
- Leased deadlines, approval expiry and schedules use backend wall clock; BEAM timer durations and callback timeouts use local runtime timing. These cannot be merged into an unstated whole-run deadline.
- A readiness report includes backend reachability, acceptance and lease evidence. Admission readiness, recovery lag, unresolved effects, upgrade metadata refresh and shutdown accounting are separate evidence, stated in the operating section.

1.19.3 Telemetry delivery

Package observations

responsibility

Emit package-owned facts about accepted actions, lifecycle, leases, graph progress, recovery and drain.

interface

fabric/telemetry event descriptors with typed metadata/measurements, run correlation and root. Sinal direct delivery or explicitly configured forwarder supplies observation route.

interactions

Execution commits precede observations. A synchronous subscriber runs in emitter process and can delay a runner; a bounded best-effort forwarder changes that latency/drop tradeoff.

invariants

Event failure cannot make a report acceptable again or roll back execution. Full action identity remains durable; turn/call subset is not globally unique. Event ordering across parent/child processes is not a total order.

failure

Observer blocking can cause RunnerBusy for commands; a withdrawn command cannot apply later. Drops and root inference have explicit events. Telemetry cannot drive child completion, scheduling or business authorization.

1.20 Verification and extension obligations

Public consumer behavior, deterministic failures and oracle provenance support claims.

1.20.1 Evidence boundaries

- Pure controllers test transitions without OTP, clocks or networking. Record suites test meaningful field combinations, malformed state, legacy fixtures and migration refusal. Neither alone establishes durable effects.
- Public runtime tests exercise both sides of effect fences, partial batches, lost acknowledgements, contested answers, cancellation/completion races, lease loss, drain and recovery. Leased memory provides deterministic ownership probes; PostgreSQL adapter scenarios prove its independent backend.
- Separate consumers complete common tasks, advanced settings, caller-owned types and failure handling through public imports. Negative graph authoring fixtures prove incompatible native connections fail to compile.
- The retained gate verifies recipe text equality between module documentation and compiled consumers, package inventory, dependency availability and deterministic consumer, service and experiment suites. It removes known live-provider credentials from child environments and never loads local credential files.
- BeamWeaver is a partial oracle, not the full product specification. Its pin, fixture rules and comparison limits are stated below. Hosted CI results, library publication and deployment drills are separate evidence.

ADR-0009 ADR-0010 ADR-0011 ADR-0016

1.20.2 Validation tools

- Nix/treefmt, Gleam and Erlang compilers, and the pinned design-layer checker own shared validation. CI invokes those tools directly. The local scripts/test.sh runs the retained packages, service integrations and compiler contract fixtures, stopping on failure.
- Repositories do not duplicate shared validation in custom registries or source inspectors. Local custom README checkers, recipe ceilings, package inventories and checker self-tests remain removed. The shared pinned Snips tool synchronizes root README excerpts from source regions; application compilers and tests own their behavior. See ADR-0018.
- Hosted workflows use locked dependencies and read-only tokens without persisted credentials. The compiler reports unavailable dependencies. Live-provider credentials remain excluded from deterministic tests.

Obligation	Check	Evidence and limit
Core lifecycle and graph design	Root test suite	Public behavior, recovery, policy, concurrency and oracle scenarios.
Usage examples	The usage examples compile and run in scripts/test.sh	Public use of bounds, failure classification, correlation, waiting, family budgets, stored reconciliation and sweeper wiring, offline against scripted models; each example is also asserted, so API drift fails the gate.
Public integrations and consumers	Per-package test suites in scripts/test.sh	Saga and Relay recipes and the application, graph, decision, jobs, writing and approver consumers.
Real PostgreSQL backend	The adapter's disposable-cluster script	A temporary cluster exercises concurrent leases, recovery, migration and retention and ignores external PostgreSQL settings.
Owned external jobs	The jobs consumer service script	A separate HTTP service with a SQLite journal, Python and Gleam tests, and cleanup.

Obligation	Check	Evidence and limit
Classifier protocol	Python unit tests of the classifier fixture	A loopback fixture, with no claim of live inference.
Writing application	The Gleam suite and the Python live-harness tests	Actual files, store restarts, scripted providers, a frozen corpus and failure accounting; live comparison stays separate.
Recipes	Compiled recipe consumers	Public composition builds and runs.
Typed graph authoring	The authoring experiment check	A fresh valid consumer and rejected input and output type mismatches.

1.20.3 Static tooling and measurement limits

- Treefmt formats Gleam, Nix, Prettier-supported files, authored Python and shell scripts. Upstream oracle captures, genuine legacy record fixtures, frozen provider and writing evidence and generated output keep their provenance and are never rewritten by tooling.
- No static security scanner, mutation suite, general architecture verifier, runtime benchmark or load certificate exists. Check durations measure verification execution only. Live-provider runs are separate explicit commands, and none replaces a deterministic test. The dated live observations, their method and their limits are retained in the measurements record.
- An unpublished coordinated API change is qualified in a normalized copy of the complete source graph and never against immutable dependency pins. Success in that copy does not qualify the recorded baseline. The generation-settings consumer uses public HTTP playback, both provider cache protocols, a directory-store restart and borrowed-client reuse, and needs no provider service or database.

1.20.4 Oracle comparison

A Oracle supplies evidence about named behavior and never authority over Fabric's contracts. The oracle ledger holds the pin, the fixture rules, the full comparison tables, the recapture procedure and the limits; this section states the rules they check.

- The oracle is the BeamWeaver fork lostbean/beam_weaver at commit d0aa1f90d31c55d49be2f7b5a24224b5e18145a1: upstream caudena/beam_weaver tag v0.1.23 (60fdcd7) plus four fork-only durability commits. The pin is by SHA alone because the fork's package version equals the Hex 0.1.23 release while its behavior differs. The license is Apache-2.0 without a NOTICE file, and fixtures come from a scratch clone, never from the fork itself.
- A fixture is a JSON file under test/oracle/fixtures holding the output of running BeamWeaver, never copied source. It records the oracle repository and commit, toolchain versions, the committed capture script, the exact command, the capture date and the license note. A capture script that transcribes BeamWeaver scenarios carries the notice that it derives from BeamWeaver under Apache-2.0 at the pinned commits.
- A fixture holds only normalized observables: transcript roles, tool call ids and names, tool message status and content, effect-log entries and outcome tags. Message UUIDs, checkpoint ids, run ids, timestamps, provider metadata and literal interrupt ids are removed at capture. Tool order inside a concurrent group compares as a multiset and transcript order compares exactly.
- A fixture is recaptured and never hand-edited, and a changed pin requires a new capture. Recapture prepares a disposable clone at the exact pin in a shell with the recorded Elixir and OTP toolchain and runs the committed capture script. The recorded command and toolchain metadata must agree with the actual run before replacement fixtures are accepted.

Category	Meaning
Faithful port	Fabric reproduces a BeamWeaver behavior and a fixture or test proves the correspondence.
Executed differential	A fixture captured by running BeamWeaver at the pin is compared by a Fabric test over a normalized observable sequence.
Inspired test	An original Fabric test motivated by a BeamWeaver test or source; it compares nothing.
Original Fabric contract	Behavior BeamWeaver lacks.
Anti-oracle	A BeamWeaver behavior Fabric deliberately rejects; the Fabric test asserts the opposite.
Unverified	Not compared, and no claim is made.

Behavior	Category	Retained evidence
Distinct tool calls execute and return in call order	Executed differential, match	two_tool_calls fixture and the oracle test
A failing or unknown tool becomes a model-visible error and the run continues	Executed differential, match with error wording excluded	tool_error_visible fixture
A model-call limit stops a looping agent	Executed differential, deliberate divergence: Fabric refuses a tool whose result could not reach another turn	model_call_limit fixture
Sub-agent start gated by review: approve runs the child once, reject never runs it	Executed differential, match with rejection wording excluded	subagent_gate_approve and subagent_gate_reject fixtures
HITL approve executes the reviewed call once; reject answers with an error and runs nothing	Executed differential, match	hitl_approve and hitl_reject fixtures
A durable pause survives the loss of every process and executes once	Executed differential, match	hitl_cold_restart fixture
Parallel tools are bounded by a concurrency limit; cancel during a tool leaves it uncertain and never retried; an uncertain effect blocks the next model turn; token budgets come from observed usage; foreign and duplicate reports are rejected	Original Fabric contract	Controller and runner tests
A crash mid-tool re-runs the tool or wedges the thread	Anti-oracle: a crash after the fence, a killed runner and a full restart record the tool as uncertain, never retried, and await reports Unattended	Runner and durable tests
Stale, duplicate or unknown-thread resume is accepted; a concurrent double resume executes twice; an invalid resume burns the pending interrupt	Anti-oracle: refusals are distinct typed outcomes, eight concurrent answers have one winner, and every refused answer leaves the request answerable	Approval tests
A child sub-agent interrupt is stringified and dropped	Anti-oracle: a child's pending approval is the parent's pending approval and answering through the parent resumes the child	Delegation tests and the application consumer
HITL edit and respond decisions, and decision-count mismatch	Unverified: Fabric has no edit or respond answer	none
Run, step and node timeouts	Unverified: per-call bounds and durable wait deadlines exist, whole-agent elapsed time does not	none

- Error and rejection wording, and final text embedding that wording, are excluded where the native protocols differ. Whole interrupt payloads, provider metadata and native identifiers are not an application authorization contract.
- Approval fixtures cover approve, reject and process-loss recovery. Sub-agent start fixtures do not establish every interrupted-child behavior, and Fabric-specific tests cover the rejected upstream hazards and managed ownership. The agent-as-graph comparisons exercise supported ordinary round trips and do not prove approval, delegation or interrupted-batch equivalence to the agent controller.
- The cold-restart capture starts separate VMs over temporary SQLite storage and a file-backed effect log, with a preload step the pinned oracle's existing-atom checkpoint decoder requires. Equality of normalized observables does not make the two storage guarantees identical.

ADR-0011 ADR-0002

1.20.5 Backend and callback extension

Extension port acceptance

responsibility

Permit variable providers, tools, verifiers, jobs, stores and graph operations while retaining common lifecycle guarantees.

interface

Public model function, typed tool binding, versioned operation, approver verifier and backend records/leased functions. Backend conformance exposes deterministic contract checks.

interactions

Each implementation is exercised from an independent package with a real native domain type/error and meaningful failure handling. Custom providers receive the same admission and budget obligations as built-in adapters.

invariants

An extension may vary business behavior while Fabric retains fencing, correlation, current context, bounded callbacks and cleanup. Captured resources belong to application lifetime and cannot be serialized.

failure

Extension authors must truthfully classify possibly-sent effects, version codec changes, bound foreign/network callbacks and preserve unreported measurements. A passing compile proves types, not every operational guarantee.

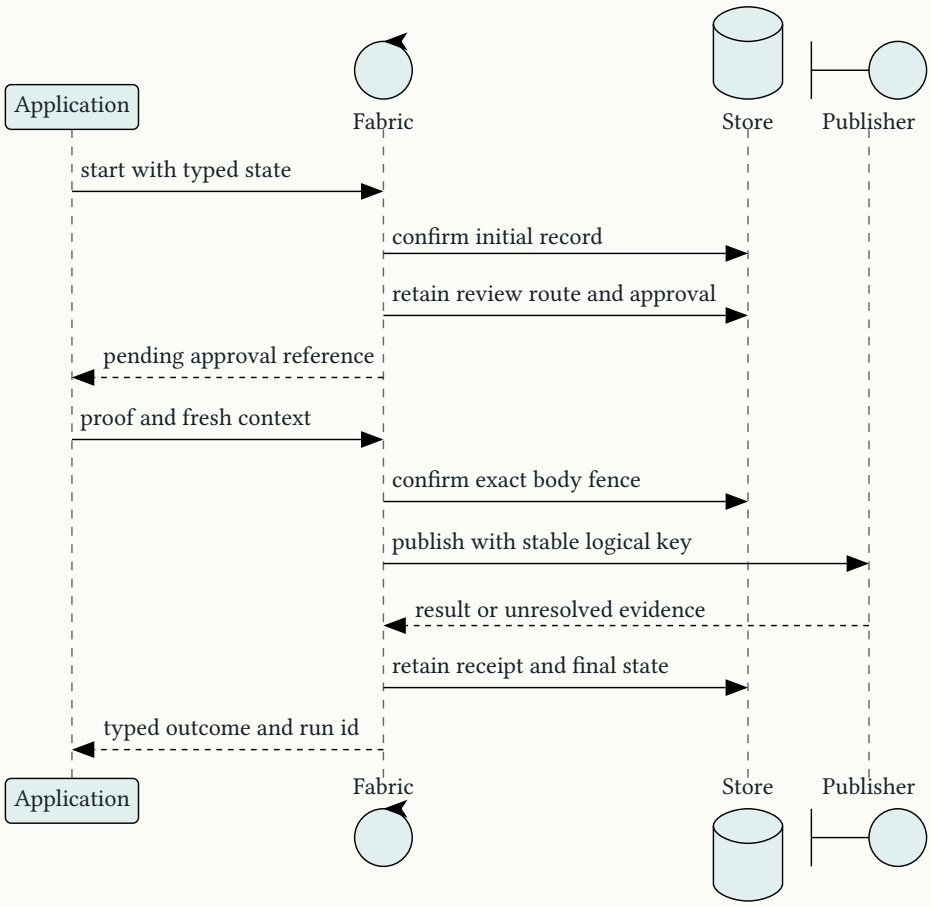
1.20.6 Retained experiments and application evidence

- experiments/graph_authoring retains synchronous native authoring and external negative consumers; production runtime authority remains in Fabric.
- experiments/workflow_composition retains three composition comparisons and a prototype patch as evidence; it is not a supported runtime dependency or a published Saga capability.
- The agent-recipe counterexample is executable in test/fabric/support/agent_recipe.gleam. It rejects unsupported action lifecycles and supplies no production replacement guarantee.
- The writing application retains a frozen corpus, artifact tools and sanitized raw captures. Review quality and runtime routing are assessed separately, and its timings are provider-inclusive observations that are not a Fabric performance claim. The comparison rationale is recorded in ADR-0012 and ADR-0017.

1.21 End-to-end walkthrough

A source-based writing graph demonstrates typed control, human ownership and restart.

SEQUENCE Committed decision before external publication



Network and credentials stay live. Every external effect follows its own permission and storage fence; observations do not control the sequence.

- An application constructs one HTTP client, shared Approvers value, typed generation/classification operations, immutable review graph and supervised store. It completes backend migrations and projection refresh before sweeper startup and admission.
- A request chooses a RunId and correlation. The graph commits initial state before selecting source input, inspecting policy and fencing the generation body. Its native generation receipt and application state commit together.
- Review returns a typed choice and retained distribution. Application acceptance chooses publish, revise or rejection from declared destinations. A revision repeats a node with a new activation until the activation/work bounds stop it.
- Publication policy requests approval. No body runs while awaiting it. A request handler verifies a current credential for the exact requirement; approval rechecks policy with fresh application context. Expiry records rejection rather than assuming permission.
- A process loss after the publish fence leaves the publish effect uncertain unless its explicit idempotency contract resolves it. Recovery reuses earlier generation and review receipts and never reruns those decisions merely because context changed.
- The operator reconciles the original publication result. A cancelled or expired graph records the evidence with no success route. Retention can remove the complete family only after its external obligations settle and the application retention window ends.

Glossary

Every term the layer declares, with the context that owns it.

Action

owned by root

One tool call or delegation identified by its run and ActionId(turn, call_id). A graph operation instead uses an activation and attempt.

Activation

owned by root

One visit to a graph node, identified by a run-local positive ordinal. Revisiting a node creates a new activation.

Agent

owned by root

A validated definition of a model-turn loop, typed tools, policy and final-answer contract.

Applied Marker

owned by root

An application-owned durable record that a logical turn's outcome was incorporated. It prevents reinvoking the turn after Fabric prunes its record.

Approval

owned by root

An action-scoped question identified by a requirement and monotonically issued revision. An answer is evidence of an authenticated decision, not a permanent grant.

Attachment

owned by root

The durable reciprocal relationship through which a parent owns a managed child or fork member.

Attempt

owned by root

One execution attempt within a graph activation. An explicitly authorized replay increases the attempt while preserving the logical activation.

Classification Wire

owned by root

The pure request encoding and receipt interpretation fixed by a classification operation version. Fresh live credentials and settings are outside it.

Context

owned by root

A live application value supplied at execution or reopening. It is not serialized into the durable run.

Correlation

owned by root

The Sinal correlation value retained at the root start and propagated through model calls, tools, operations and children. It is an observation join key, not a replay identity.

Definition Version

owned by root

The application assertion that deployed behavior and codecs retain a meaning. Structural manifests cannot detect arbitrary callback changes.

Discovery

owned by root

A versioned scheduling projection of idle dependencies, polls and absolute due times. It is a wakeup hint, never effect authority.

Drain

owned by root

The bounded shutdown window in which runners finish started work, save results and hand off the remainder. A runner that exceeds the window is killed.

Effect Fence

owned by root

The confirmed durable transition from queued work to started work. It precedes calling a body that may affect an external system.

Family

owned by root

A root execution and all executions joined through validated parent attachments. Its retention and optional budget span agent and graph runs.

Family Budget

owned by root

The immutable limits a root declares for cumulative work admissions, child starts and nesting depth, enforced through the family ledger.

Family Grant

owned by root

One retained reservation in the root budget ledger. Its logical claim identity prevents the same work from spending capacity twice.

Fork

owned by root

One activation-scoped ordered membership of private child graphs. The parent alone owns joining their results into parent state.

Graph

owned by root

A validated immutable protocol over typed application state, operations and declared routes.

Incarnation

owned by root

The increasing ownership generation retained when lost work is recovered. A report from an older incarnation has no authority over current work.

Invocation

owned by root

A protocol-neutral bounded request to start or reopen an agent or graph run. Keyed lifetime and caller-owned unkeyed lifetime differ.

Job

owned by root

An external computation identified by a retained receipt. Observing it, requesting a stop and proving terminal completion are distinct contracts.

Lease

owned by root

A backend-clock ownership claim that determines which store drives a run. A revision check separately decides whether a record write may commit.

Node

owned by root

A stable identifier of one live VM that shares a database backend. Two live VMs never share one node id. *Avoid*: process name, which is unique only within one VM.

Oracle

owned by root

A pinned external implementation whose captured, normalized behavior is compared with Fabric for named behavior only. It supplies evidence and never authority.

Original input

owned by root

The immutable ordered historical messages and current user prompt supplied to one agent run. *Avoid*: conversation resource, which implies an independently managed lifecycle; first message, which excludes later imported history and the current prompt.

Policy

owned by root

The application decision that allows an exact decoded action, denies it, or requires approval with current context.

Proof

owned by root

Short-lived evidence minted by one configured Approvers value after authenticating and authorizing a credential for one requirement.

Readiness

owned by root

A report of backend reachability, admission acceptance and local runner lease safety at the instant after one bounded read. It promises nothing about later requests.

Receipt

owned by root

A committed graph activation result containing its input, output, resulting application state and route. It fixes a past decision independently of later inference.

Recipe

owned by root

Application code compiled verbatim from public APIs that translates an independent package's outcomes into Fabric ports. Core depends on none of those packages.

Reconciliation

owned by root

An explicit assertion of the original action or operation result, checked against its retained identity and contract.

Replay Contract

owned by root

An explicit application promise, declared on a tool or operation, that bounded re-execution of an interrupted body is safe. Without it an interrupted body stays an uncertain effect.

Run

owned by root

A stored execution named by a caller-chosen RunId, or for a managed child by a RunId Fabric derives from its parent attachment. Its durable data belongs to either the agent controller or the graph controller.

Settled Family

owned by root

A complete mutually linked family with no unresolved owned effects or cleanup. Terminal execution status alone does not establish settlement.

Settlement

owned by root

Late result delivery for a stopped or uncertain tool. It supplies evidence for the same invocation and does not start another effect.

Signal

owned by root

A versioned typed value delivered to one committed graph wait. Its reference correlates a command but grants no authorization.

Sweeper

owned by root

A supervised scanner that finds expired leases and changed or due idle waits and recovers each through a registered compatible root.

Uncertain Effect

owned by root

An effect whose actual outcome is unresolved. Loss of a local task does not establish that its external effect stopped.

Write Token

owned by root

A fresh token embedded in one encoding. Exact bytes and target revision identify the writer when an acknowledgement is lost.